DÉMONSTRATEUR WI-FI 6

ObCP + X-NUCLEO-67W61M1

Guide d'installation et d'utilisation des démonstrations 1 à 4

› Démo 1 : scanner Wi-Fi et suivi du signal

› Démo 2 : connexion et services réseau (DHCP, DNS, ping, SNTP)

› Démo 3 : serveur web en point d'accès

› Démo 4 : télémétrie MQTT

› Sur batterie : démarrage automatique du serveur web et de MQTT

Programme version 1.2.0 (projet STM32CubeMX) — module ST67W611M1, firmware mission T01 v2.0.x

28 octobre 2026

Sommaire

1. Présentation 4
1.1 Les quatre démonstrations 4
1.2 Différences avec la version NUCLEO-L476RG 4
1.3 Architecture 5
1.4 Contenu du paquet 5
1.5 État de validation 6

2. Matériel et montage 8
2.1 Matériel nécessaire 8
2.2 Repères sur l'ObCP 8
2.3 Montage et mise sous tension 9
2.4 Brochage utilisé 9
2.5 LED RVB, sortie PWM et accéléromètre de l'ObCP 10

3. Firmware du module ST67W611M1 11
3.1 Version nécessaire et fichiers 11
3.2 Vérifier le firmware présent 11
3.3 Mise à jour depuis le démonstrateur (module ota) 11
3.4 Première programmation par l'ObCP (module boot) 13
3.5 Première programmation avec une NUCLEO-H563ZI 14

4. Programmation de l'ObCP et compilation 16
4.1 Le chargeur USB DFU 16
4.2 Programme précompilé avec STM32CubeProgrammer 16
4.3 Avec STM32CubeIDE 16
4.4 Modifier la configuration avec STM32CubeMX 17
4.5 En ligne de commande 20
4.6 Ressources utilisées 20

5. Prise en main du terminal 21
5.1 Le port COM virtuel de l'ObCP 21
5.2 Réglages du terminal 21
5.3 Premier démarrage 21
5.4 Utilisation de la ligne de commande 22
5.5 Configuration enregistrée 23
5.6 Démarrage automatique (utilisation sur batterie) 24

6. LED RVB, sortie PWM et accéléromètre 27
6.1 LED RVB 27
6.2 Sortie PWM 27
6.3 Accéléromètre et capteurs 27

7. Démo 1 – Scanner Wi-Fi et suivi du signal 28
7.1 Balayage des réseaux 28
7.2 Occupation des canaux 28
7.3 Suivi du signal en direct 29

8. Démo 2 – Connexion et services réseau 30
8.1 Connexion à un réseau 30
8.2 Adressage IP fixe ou DHCP 30
8.3 DNS, ping et heure 31
8.4 Ce qui se passe en tâche de fond 31

9. Démo 3 – Serveur web en point d'accès 32
9.1 Démarrage 32
9.2 La page web 33
9.3 Interface HTTP 35

10. Démo 4 – Télémétrie MQTT 37
10.1 Principe 37
10.2 Démarrage 37
10.3 Topics et format des messages 38
10.4 Commandes à distance 38
10.5 Côté PC 39

10.6 Utiliser un autre broker 39
11. Référence des commandes 40
12. Dépannage 41
13. Pour aller plus loin 44
Annexe A – Organisation du logiciel 45
Annexe B – Protocole SPI du ST67W611M1 46
Annexe C – Tests sur PC 46
Annexe D – Références et licences 48

1. Présentation

Ce paquet contient un programme de démonstration complet pour l'**ObCP** (Objet Connecté Personnel de SUPMICROTECH-ENSMM, STM32L476RG) équipé de la carte d'extension **X-NUCLEO-67W61M1** (module Wi-Fi 6 / Bluetooth LE **ST67W611M1**). Il reprend les quatre démonstrations de la version NUCLEO-L476RG et les adapte à l'ObCP : la console passe par le **port USB de l'ObCP** (port COM virtuel), le programme se charge par le **chargeur USB DFU** de ST, et la LED RVB, la sortie PWM et l'accéléromètre de la carte sont mis à contribution. Tout se pilote depuis un terminal série, sans recompiler le programme.

Version 1.1 : le projet est désormais décrit par un fichier STM32CubeMX, `ObCP_WiFi_Demo.ioc`. L'initialisation du matériel est générée par CubeMX ; on peut modifier la configuration (broches, périphériques, horloges) et **régénérer le code sans casser la démo** (4.4).

Version 1.2 : pour une utilisation **sur batterie, sans terminal**, la carte peut lancer seule, à la mise sous tension, la connexion à la box enregistrée, le **point d'accès et le serveur web** et la **télémétrie MQTT** (commande `auto`, 5.6). Le code est commenté pour un lecteur débutant et s'accompagne d'une **documentation du programme** : organisation, schémas de déroulement, référence de toutes les fonctions (`Doc/Documentation_Programme_ObCP.docx`).

1.1 Les quatre démonstrations

Démo	Ce que l'on montre	Commandes principales
1. Scanner Wi-Fi et suivi du signal	Réseaux visibles (RSSI, canal, sécurité, Wi-Fi 6), occupation des canaux 1 à 13, suivi du RSSI en direct (couleur de la LED RVB selon le signal, export CSV)	`scan`, `channels`, `track`
2. Connexion et services réseau	Connexion à une box, DHCP ou IP fixe, résolution DNS, ping, mise à l'heure de la RTC par SNTP (fuseau, heure d'été)	`connect`, `status`, `ip`, `dns`, `ping`, `ntp`, `time`
3. Serveur web en point d'accès	L'ObCP crée son propre réseau Wi-Fi (QR code de connexion) et sert un tableau de bord : accéléromètre (bille qui roule), couleur de la LED, sortie PWM, réseaux voisins, configuration Wi-Fi depuis le téléphone	`web start`, `web status`, `web stop`
4. Télémétrie MQTT	Publication périodique d'un état JSON (accélérations, inclinaisons, LED, PWM…) vers un broker, commandes à distance (LED, couleur, PWM, période…), évènements boutons	`mqtt start`, `mqtt pub`, `mqtt sub`

S'y ajoutent les commandes propres à l'ObCP : `led` (couleur et luminosité de la LED RVB), `pwm` (sortie du bornier) et `sensors` (accéléromètre, inclinaisons), présentées au chapitre 6.

1.2 Différences avec la version NUCLEO-L476RG

Élément	NUCLEO-L476RG	ObCP
Console	Port COM virtuel du ST-LINK (USART2, 115200 bauds)	Port COM virtuel de l'ObCP lui-même (USB, classe CDC) : débit indifférent (5.1)
Programmation	ST-LINK intégré : glisser-déposer, SWD	Chargeur USB DFU de ST : interrupteur Prog/Run et Reset, ou commande `dfu` (4.1)
Débogage pas à pas	ST-LINK intégré	Sonde SWD externe sur le connecteur Debug J17, facultatif (4.3)

Élément	NUCLEO-L476RG	ObCP
LED	LED externe facultative sur PC8	LED RVB D5 intégrée : couleur et luminosité (6.1)
Mesure	Potentiomètre facultatif sur A0	Accéléromètre 3 axes intégré : accélérations, inclinaisons (6.3)
Sortie de puissance	–	Sortie PWM sur le bornier J13 : 5 V, 2,5 W au plus (6.2)
Bouton	B1 (bleu)	User1 (SW1 de l'ObCP)
Horloge	MSI asservie au quartz 32 kHz	Quartz 8 MHz (HSE), nécessaire à l'USB
Première programmation du module	Nucleo prise en charge par ST ou adaptateur USB-UART	Par l'ObCP elle-même : `module boot` et QConn_Flash, sans autre matériel (3.4)
Alimentation	USB du ST-LINK	USB de l'ObCP ou batterie lithium ; interrupteur SW2

Le pilote du module (`Core/Src/st67w6x`), les démos et la mise à jour du module par le terminal (`module ota`) sont identiques à ceux de la version NUCLEO-L476RG : le brochage Arduino de l'ObCP est le même que celui de la Nucleo.

1.3 Architecture

Le module ST67W611M1 exécute le firmware « mission T01 » : la pile TCP/IP (LwIP), le client DHCP, le DNS, le SNTP, les sockets et le client MQTT tournent **dans le module**. Le STM32L476 de l'ObCP dialogue avec lui par **commandes AT** transportées dans des trames **SPI** (signaux CS, RDY, CHIP_EN et BOOT en plus du bus SPI). Côté PC, l'ObCP se présente comme un port COM virtuel USB. Le programme est écrit sans RTOS : une boucle principale traite la console et exécute les tâches de fond (évènements Wi-Fi, reconnexion, NTP, serveur web, MQTT, boutons).

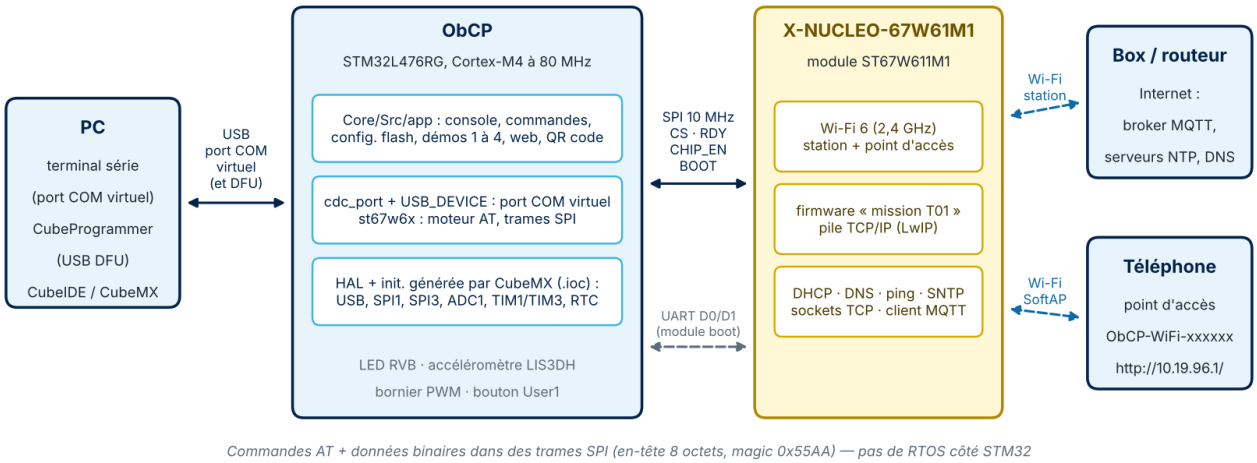


Figure 1 – Architecture matérielle et logicielle du démonstrateur sur l'ObCP

1.4 Contenu du paquet

Dossier / fichier	Contenu
`ObCP_WiFi_Demo.ioc`	Configuration STM32CubeMX : broches et leurs étiquettes, horloges, SPI1, SPI3, ADC1, TIM1, TIM3, RTC, USB (classe CDC), interruptions (4.4)

Dossier / fichier	Contenu
`Core/Src/`, `Core/Inc/` : fichiers générés	Initialisation générée par CubeMX : `main.c`, `gpio.c`, `spi.c`, `adc.c`, `tim.c`, `rtc.c`, `stm32l4xx_it.c`, `stm32l4xx_hal_msp.c`, `main.h`… La démo n'y occupe que des sections USER CODE
`Core/Src/` : fichiers de la démo	`board.c` (chargeur DFU, quartz 32 kHz, fin de l'initialisation, diagnostic, RTC), `cdc_port.c` (port COM virtuel), `fault_handlers.c` (fautes du processeur), `uart_bridge.c` (passerelle module boot), `ncp_port_obcp.c` (adaptation du pilote)
`Core/Src/app/`	Application : console, commandes, configuration en flash, démarrage automatique, LED RVB (`color.c`), accéléromètre (`lis3dh.c`), démos 1 à 4, page web, QR code, mise à jour du module
`Core/Src/st67w6x/`	Pilote « bare-metal » du ST67W611M1 : trames SPI, moteur AT, Wi-Fi, réseau, MQTT, mise à jour (portable : seul `ncp_port.h` dépend de la carte)
`Core/Src/qrcodegen/`	Génération de QR codes (Project Nayuki, licence MIT)
`Core/Inc/`	Tous les en-têtes, générés et de la démo
`USB_DEVICE/`	Généré par CubeMX : USB device, classe CDC ; les rappels sont transmis à `cdc_port.c`
`Drivers/`, `Middlewares/ST/`	Bibliothèques copiées par CubeMX : HAL STM32L4, CMSIS, bibliothèque USB device de ST (paquet STM32Cube FW_L4 V1.18.1)
`Tests/`	Tests sur PC : pilote, modules, port COM virtuel, simulation complète de l'application (`make -C Tests`)
`Tools/verif_cubemx.py`	Vérification du projet après une régénération par CubeMX (4.4)
`Tools/rssi_logger.py`	Enregistrement du RSSI en CSV et tracé de la courbe (démo 1)
`Tools/maj_module_ota.py`	Mise à jour du firmware du module depuis le PC, par le terminal (3.3)
`Binaries/`	Programme précompilé (`.hex`, `.bin`, `.elf`) à charger par USB DFU avec STM32CubeProgrammer (4.2)
`.project`, `.cproject`, `.mxproject`, `Makefile`	Projet STM32CubeIDE (configurations Debug et Release), mémoire de CubeMX et compilation en ligne de commande
`Doc/`	Ce guide, et la documentation du programme (`Documentation_Programme_ObCP.docx`) : organisation du code, schémas de déroulement, dialogue avec le module, mémoire, référence des fonctions, recettes pour modifier le programme

1.5 État de validation

La **version 1.0**, dont l'initialisation du matériel était écrite à la main, a été essayée sur ObCP : elle fonctionne.

Dans la **version 1.1**, l'application, le pilote du module et les démos sont inchangés ; seule l'initialisation du matériel a été reprise sous la forme que génère STM32CubeMX 6.11.1 (la version du projet modèle de l'ObCP), avec les mêmes réglages : 80 MHz, USB à 48 MHz, SPI1 en mode 0 à 10 Mbit/s, SPI3 en mode 3 à 5 Mbit/s, PWM à 1 kHz, RTC sur le quartz 32 kHz, mêmes priorités d'interruption. Cette version a été :

- **compilée sans erreur ni avertissement** avec le Makefile (GCC 15, options `-Wall -Wextra`) et avec les réglages des configurations STM32CubeIDE (Debug en -Og et en -O0, Release), y compris avec les seuls chemins d'inclusion et dossiers sources que déclare CubeMX ;
- **comparée au code généré par CubeMX** : la partie générée des fichiers (hors sections USER CODE) a été confrontée à celle du projet modèle de l'ObCP, produite par la même version de CubeMX ; les différences viennent toutes de la configuration (broches, périphériques, horloges) ;

- **testée sur PC** : le pilote face à un module ST67W611M1 simulé (193 vérifications), les modules indépendants du matériel (313 vérifications : calendrier, JSON, HTTP, configuration, réception YMODEM, couleurs de la LED, accéléromètres LIS3DH et LIS3DSH simulés, fin de session de la passerelle `module boot`), le port COM virtuel face à une classe CDC simulée (115 vérifications) et l'application complète avec les quatre démos (284 vérifications) ; la page web a été manipulée dans un navigateur (Chromium) et la mise à jour du module par le terminal essayée de bout en bout sur PC.

La version 1.1 a ensuite été **essayée sur une ObCP réelle**, avant et après une **régénération effective par STM32CubeMX** suivie d'une recompilation : tout fonctionne.

La **version 1.2** ajoute le démarrage automatique du point d'accès et du serveur web (`web.auto`), la commande `auto` et la carte « Démarrage automatique » de la page web, ainsi que des commentaires détaillés dans tout le code de la démo ; l'initialisation du matériel (fichiers générés par CubeMX) n'a pas changé. Cette version a été **compilée sans erreur ni avertissement** (Makefile, configurations Debug et Release de STM32CubeIDE) et **testée sur PC** : modules indépendants du matériel (325 vérifications, dont la relecture d'une configuration enregistrée par la version 1.1), application complète (284 vérifications) et trois mises sous tension simulées avec une configuration enregistrée : box, serveur web et MQTT ; serveur web sans box ; box qui refuse la connexion (87 vérifications). Elle **n'a pas encore été essayée sur une ObCP réelle**.

> **Attention :** premier essai conseillé de la version 1.2 : programmer l'ObCP par DFU (4.2), vérifier la bannière (version 1.2.0, ligne « Démarrage »), connecter la carte à la box (`connect`), puis `auto web on`, `auto mqtt on` et `save` ; alimenter ensuite la carte par sa batterie, sans câble USB : le point d'accès ObCP-WiFi-<id> doit apparaître, la page web s'ouvrir et la télémétrie arriver sur le broker (5.6). Si la LED rouge clignote dès la mise sous tension, voir le chapitre 12.

2. Matériel et montage

2.1 Matériel nécessaire

- Une carte **ObCP** (version 2.3, « ObCP2 option CROC », ou ultérieure) et une carte **X-NUCLEO-67W61M1**.
- Un câble USB **de données** adapté au connecteur USB (U6) de l'ObCP : certains câbles ne servent qu'à la charge et n'ont pas les fils de données.
- Un PC avec **STM32CubeProgrammer** (programmation par USB DFU), un terminal série (PuTTY, Tera Term, picocom…) et, pour modifier le programme, STM32CubeIDE ou une chaîne GCC Arm.
- Un point d'accès Wi-Fi **2,4 GHz** (box, routeur de TP ou partage de connexion d'un téléphone) et un téléphone pour la démo 3.
- Facultatif : une batterie lithium (connecteur J1) pour une utilisation sans câble, avec démarrage automatique des services (5.6) ; une charge 5 V à piloter sur le bornier PWM (petit moteur, ventilateur, bande de LED…, 2,5 W au plus).
- Pour une première programmation du module (chapitre 3) : rien de plus, l'ObCP sert de passerelle (3.4). Une NUCLEO-H563ZI (ou U575ZI-Q, H7S3L8, N657X0-Q) peut servir de solution de repli (3.5).

2.2 Repères sur l'ObCP

La figure ci-dessous situe les éléments utilisés par le démonstrateur (vue 3D de l'ObCP V2.3 issue du dépôt du projet ; les repères sont ceux de la sérigraphie et du schéma).

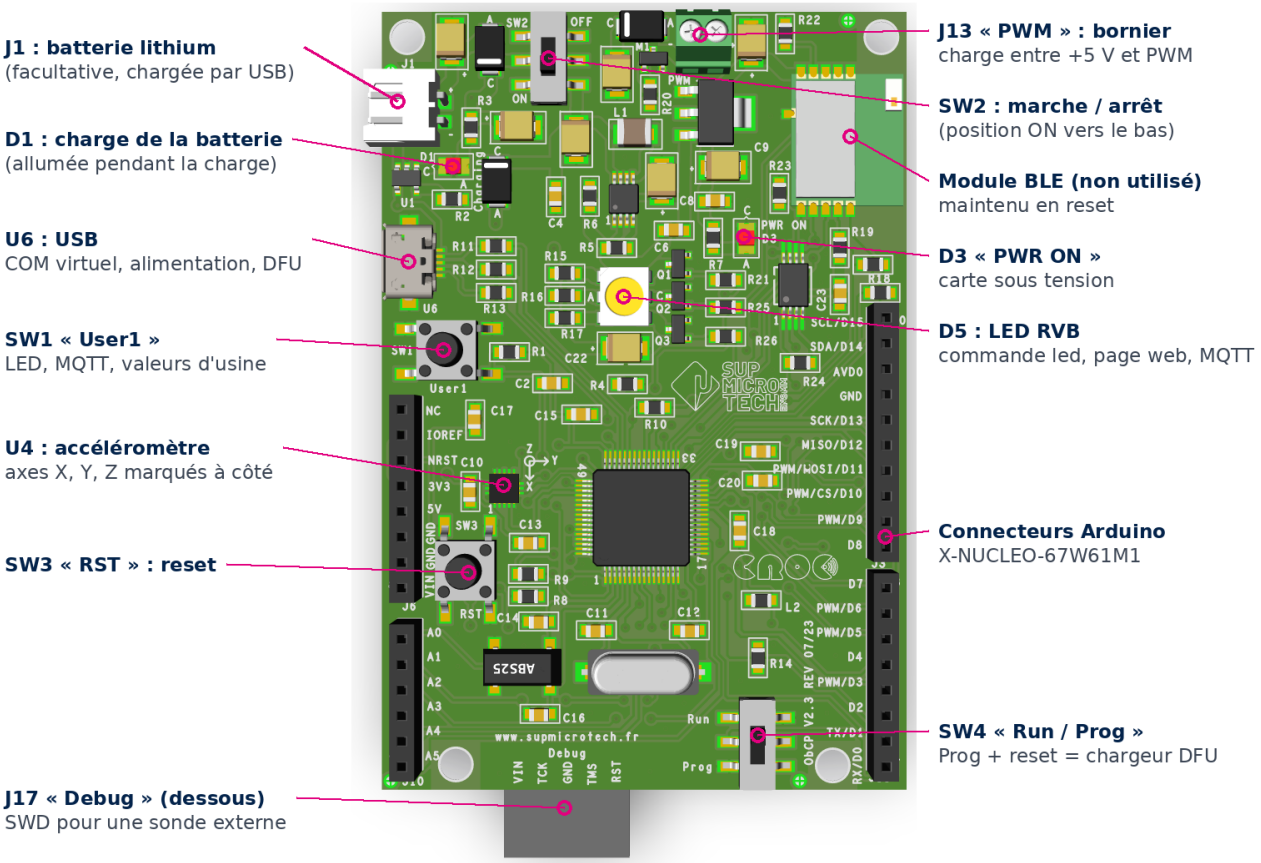


Figure 2 – Éléments de l'ObCP utilisés par le démonstrateur

2.3 Montage et mise sous tension

1. Vérifier que les cavaliers **JP1** et **JP2** de la X-NUCLEO-67W61M1 sont en place : ils alimentent le module (ils servent à mesurer les courants VDD_IO et VDD_RF).
2. Placer l'interrupteur **SW4** sur **Run**. Une fois la carte d'extension enfichée, SW4 et le bouton **RST** (SW3) peuvent devenir peu accessibles : la commande `dfu` permet alors de reprogrammer l'ObCP sans y toucher (4.1).
3. Enficher la X-NUCLEO-67W61M1 sur les connecteurs Arduino de l'ObCP (J3, J6, J10, J11), sans décalage de broches.
4. Relier le connecteur USB de l'ObCP au PC, puis placer l'interrupteur d'alimentation **SW2** sur **ON** : la LED « PWR ON » (D3) s'allume. L'USB alimente les deux cartes et fournit le port COM virtuel ; une batterie branchée sur J1 se charge (LED D1 allumée pendant la charge).

> **Bon à savoir :** sur batterie seule (SW2 sur ON, sans USB), le terminal n'est pas accessible, mais le démonstrateur démarre normalement : il rejoint le réseau Wi-Fi enregistré et, si `mqtt.auto` vaut 1, publie sa télémétrie (démo 4), de quoi suivre l'accéléromètre à distance. Les pointes de courant du Wi-Fi sollicitent la batterie : en cas de redémarrages inopinés, alimenter par l'USB.

2.4 Brochage utilisé

Fonction	Arduino	STM32L476	Remarque
SPI horloge / MISO / MOSI	D13 / D12 / D11	PA5 / PA6 / PA7	SPI1, 10 MHz, mode 0
SPI CS	D10	PB6	Actif à l'état haut, réveille le module
SPI RDY	D3	PB3	Entrée : le module demande / accepte un échange (allume aussi LD1 sur la X-NUCLEO)
CHIP_EN	D5	PB4	Mise en marche / redémarrage du module
BOOT	D6	PB10	0 = fonctionnement normal, 1 = mode téléchargement
Bouton SW1 (X-NUCLEO)	D8	PA9	Actif bas
UART du module	D0 / D1	PA3 / PA2	USART2, utilisé seulement par `module boot` (3.4) ; entrées le reste du temps
Console (USB, U6)	–	PA11 / PA12	USB device, classe CDC (port COM virtuel)
LED RVB D5	–	PC6 / PC8 / PC9	TIM3_CH1 (rouge), CH3 (vert), CH4 (bleu), 1 kHz
Sortie PWM (J13)	–	PB1	TIM1_CH3N, 1 kHz, transistor MOS IRLML2803
Accéléromètre U4	–	PC10 / PC11 / PC12, CS : PC5	SPI3, mode 3, 5 MHz
Bouton User1 (SW1 de l'ObCP)	–	PC13	Actif bas
Module BLE	–	PC2 (reset), PB2 (CS)	Non utilisé : maintenu en reset
Quartz	–	PH0 / PH1, PC14 / PC15	8 MHz (horloge système et USB), 32,768 kHz (RTC)

> **Bon à savoir :** le brochage Arduino de l'ObCP reprend celui de la NUCLEO-L476RG : le pilote du module est inchangé. Contrairement à la Nucleo, les lignes D0/D1 sont reliées directement au STM32 (PA3/PA2) : le

programme les laisse en entrées, sauf pendant `module boot` où elles relient le PC à l'UART du module par l'USART2.

PB3 sert de signal RDY : la sortie de trace SWO n'est pas utilisable (le débogage SWD par J17 reste possible).

Attention : le ST67W611M1 fonctionne uniquement dans la bande des **2,4 GHz**. Un réseau 5 GHz seul n'apparaît pas dans le balayage : activer la bande 2,4 GHz de la box si nécessaire.

2.5 LED RVB, sortie PWM et accéléromètre de l'ObCP

- **LED RVB D5** : trois LED (rouge, verte, bleue) commandées en PWM à 1 kHz par TIM3, à travers des transistors 2N2222. Le programme règle la couleur (0 à 255 par composante) et la luminosité (0 à 100 %, avec une correction gamma pour une progression régulière à l'œil).
- **Sortie PWM J13** : bornier à vis piloté par le transistor MOS IRLML2803 (TIM1_CH3N, 1 kHz). La charge se branche entre la borne +5 V (broche 2 de J13) et la borne PWM (broche 1) : le transistor la relie à la masse pendant la durée du rapport cyclique. Une diode de roue libre (D4) protège le transistor avec une charge inductive (moteur, relais). Puissance maximale : 2,5 W sous 5 V, soit 0,5 A.
- **Accéléromètre U4** : 3 axes, liaison SPI3, ±2 g, 50 mesures par seconde. Les axes sont gravés à côté du composant : X vers le bord du connecteur Debug (J17), Y vers les connecteurs Arduino D0 à D15, Z vers le haut. Carte posée à plat, composants vers le haut : Z ≈ +1 g, X et Y ≈ 0.

Bon à savoir : la nomenclature de fabrication de l'ObCP indique un **LIS3DSH**, alors que le programme modèle de l'ObCP pilote un **LIS3DH** (registres et protocole SPI différents). Le démonstrateur reconnaît les deux au démarrage (registre WHO_AM_I : 0x33 pour le LIS3DH, 0x3F pour le LIS3DSH) et affiche le modèle trouvé dans la bannière et avec `sensors`.

3. Firmware du module ST67W611M1

3.1 Version nécessaire et fichiers

Le module possède son propre firmware, indépendant du programme du STM32. Ce démonstrateur nécessite le firmware **« mission T01 » version 2.0.x** (pile TCP/IP dans le module) ; le firmware « T02 » (pile TCP/IP sur l'hôte) ne convient pas. STMicroelectronics publie ces firmwares dans le dépôt GitHub de X-CUBE-ST67W61 (version 1.3.0 : 2.0.89, 2.0.97 et **2.0.106**, la plus récente étant conseillée).

Inutile de télécharger tout le paquet (plus de 300 Mo) : selon la situation, un seul fichier ou un seul dossier suffit.

Situation	Ce qu'il faut	Où le trouver
Module déjà en T01 2.0.x : mise à jour depuis le démonstrateur (3.3)	Un fichier : `st67w611m_mission_t01_v2.0.106.xz.ota` (860 Ko)	Téléchargement direct — dossier NCP Binaries
Première programmation : module d'usine trop ancien, en T02 ou sans firmware (3.4)	Une quinzaine de fichiers du dossier `Projects/ST67W6X_Scripts` (outil QConn_Flash, binaires), environ 30 Mo	Dossier ST67W6X Scripts ; liste des fichiers en 3.4

Les modules sont programmés en usine avec un firmware mission, mais celui-ci peut être ancien : sur le forum ST, un module répondant avec un en-tête SPI 0x5501 au lieu de 0x55AA contenait un binaire antérieur aux architectures T01/T02 et devait être reprogrammé.

3.2 Vérifier le firmware présent

Au démarrage, le démonstrateur affiche la version du firmware du module (bannière décrite en 5.3) ; la commande `module` la réaffiche. Les messages suivants indiquent qu'une première programmation est nécessaire (3.4) :

Message au démarrage	Signification
« en-tête invalide (magic 0x5501 au lieu de 0x55AA) »	Firmware trop ancien (antérieur aux architectures T01/T02)
« firmware T02 détecté : programmer le binaire mission T01 »	Firmware de l'autre architecture (pile TCP/IP sur l'hôte)
« le module ne répond pas »	Carte mal enfichée, cavaliers JP1/JP2 absents, ou module sans firmware utilisable

Si le module démarre normalement avec une version 2.0.x plus ancienne que celle souhaitée, la mise à jour se fait directement depuis le démonstrateur (3.3).

3.3 Mise à jour depuis le démonstrateur (module ota)

Quand le module exécute déjà un firmware mission T01 2.0.x, le démonstrateur le met à jour lui-même, **sans câblage ni autre carte** : le fichier est envoyé depuis le PC par le terminal (protocole **YMODEM**), le STM32 le transmet au module par le lien SPI (commandes AT+OTASTART, AT+OTASEND et AT+OTAFIN, comme l'exemple FOTA de ST) ; le module vérifie l'image (empreinte SHA-256 contenue dans son en-tête), puis redémarre dessus. Le STM32 ne stocke pas l'image : chaque bloc reçu est aussitôt transmis au module.

Étape 1 – Télécharger l'image

- Image conseillée : st67w611m_mission_t01_v2.0.106.xz.ota (compressée, 860 Ko).
- Variante non compressée : st67w611m_mission_t01_v2.0.106.bin.ota (1,5 Mo, transfert plus long).

- Si une version plus récente est publiée, la prendre dans le dossier NCP_Binaries : seuls les fichiers `st67w611m_mission_t01_v….ota` conviennent. Les `.bin` sans `.ota` (première programmation) et les images `t02` ou `mfg` sont refusés par la carte.

Étape 2 – Envoyer le fichier

Trois façons, au choix (COM7 désigne ici le port COM virtuel de l'ObCP, voir 5.1) :

- **Tera Term** : taper `module ota` dans le terminal, puis menu **Fichier › Transfert › YMODEM › Envoyer…** (Tera Term en anglais : File › Transfer › YMODEM › Send…) et choisir le fichier. Avant le transfert, des « C » s'affichent (le démonstrateur demande le fichier toutes les 2 s) ; Tera Term affiche ensuite une fenêtre de progression. PuTTY ne sait pas envoyer de fichier.
- **Script fourni** (Windows, Linux, macOS ; Python 3 et pyserial) : fermer le terminal série, puis lancer la commande ci-dessous ; le script tape lui-même `module ota`, envoie le fichier en affichant la progression, puis le résultat.

```
pip install pyserial
python Tools/maj_module_ota.py --port COM7 st67w611m_mission_t01_v2.0.106.xz.ota
```

- **Linux / macOS avec lrzsz** : `picocom --send-cmd "sz -vv --ymodem -k" /dev/ttyACM0`, taper `module ota`, puis Ctrl-A Ctrl-S et le nom du fichier.

Étape 3 – Installation

À la fin du transfert, le démonstrateur affiche le fichier reçu, demande l'installation au module, attend son redémarrage (jusqu'à 40 s), le réinitialise et affiche la nouvelle version :

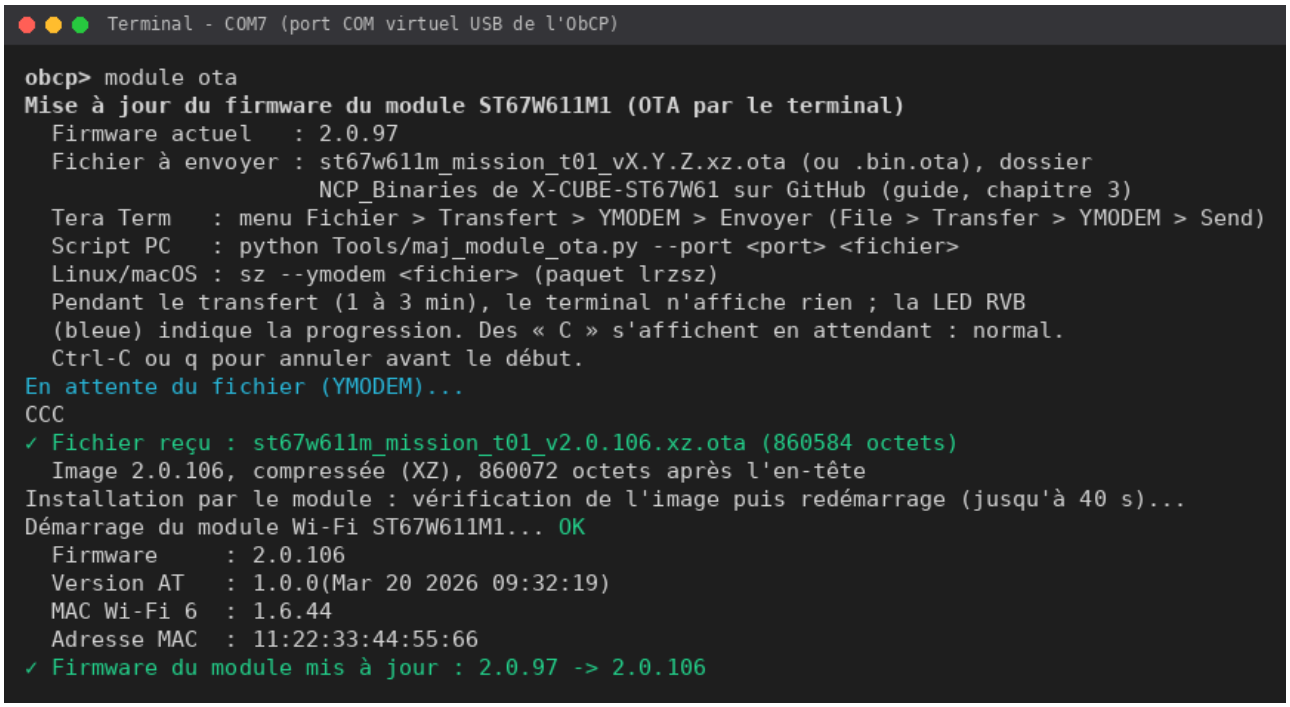


Figure 3 – Mise à jour du module par le terminal (commande module ota, fichier envoyé en YMODEM)

Sur la NUCLEO-L476RG, la liaison à 115200 bauds limitait le transfert de l'image compressée à environ 75 s. Le port COM virtuel de l'ObCP n'a pas cette limite : le transfert devrait être plus court (durée non encore mesurée sur carte). La liaison Wi-Fi et MQTT se rétablissent seules ; le point d'accès de la démo 3 est à relancer (`web start`). Pendant le transfert, la LED RVB (bleue) indique la progression et le terminal reste muet.

> **Bon à savoir :** en cas d'erreur avant l'installation (transfert interrompu, fichier refusé, Ctrl-C ou q pour annuler avant le début), la session est abandonnée et **le module conserve son firmware**. Le module refuse aussi une image altérée (empreinte SHA-256 fausse) ; une version plus ancienne que celle en place peut être refusée.

Ne pas couper l'alimentation pendant la phase « Installation par le module ».

Attention : `module ota` ne peut pas réparer un module d'usine trop ancien (en-tête 0x5501) ni un module en T02 : ces modules ne comprennent pas les commandes de mise à jour du démonstrateur. Dans ce cas, utiliser 3.4.

3.4 Première programmation par l'ObCP (module boot)

La première programmation passe par le chargeur en ROM du module, son UART et l'outil **QConn_Flash** de ST. Sur l'ObCP, l'UART du module (lignes D0/D1) est relié au STM32 : la commande `module boot` place le module en mode téléchargement, puis l'ObCP **relaie son port COM virtuel vers l'UART du module**. QConn_Flash, lancé sur le PC avec ce port, dialogue alors directement avec le module ; aucun adaptateur ni autre carte n'est nécessaire. Le débit suit celui que choisit QConn_Flash (500 kbit/s puis 2 Mbit/s).

Étape 1 – Préparer les fichiers sur le PC

- **Récupération la plus simple** (environ 300 Mo) : sur la page GitHub de X-CUBE-ST67W61, bouton **Code › Download ZIP**, puis décompresser ; seul le dossier `Projects\ST67W6X_Scripts` sert.
- **Récupération légère** (environ 30 Mo) : télécharger un à un les fichiers ci-dessous depuis les dossiers ST67W6X_Scripts/Binaries et ST67W6X_Scripts/LittleFS/littlefs (ouvrir le fichier sur GitHub, puis icône **Download raw file**), en recréant exactement cette arborescence : QConn_Flash trouve les fichiers par leur chemin relatif.

```
ST67W6X_Scripts\
├─ Binaries\
│   ├─ NCP_Binaries\
│   │   ├─ mission_t01_flash_prog_cfg.ini
│   │   ├─ st67w611m_boot2_v8.1.9.bin
│   │   ├─ partition.bin
│   │   ├─ efusedata.bin
│   │   └─ st67w611m_mission_t01_v2.0.106.bin
│   └─ QConn_Flash\
│       ├─ QConn_Flash_Cmd.exe            (Linux : QConn_Flash_Cmd-ubuntu)
│       ├─ chips\qcc743\eflash_loader\eflash_loader_cfg.conf
│       ├─ chips\qcc743\efuse_bootheader\efuse_bootheader_cfg.conf
│       ├─ chips\qcc743\efuse_bootheader\flash_para.bin
│       └─ utils\pem\room_root_publickey_ecc.pem
└─ LittleFS\littlefs\littlefs.bin
```

Attention : ne prendre que le binaire **mission t01** (`st67w611m_mission_t01_v2.0.106.bin`, sans `.ota`) : ni `t02`, ni `mfg`.

Étape 2 – Préparer le fichier de configuration

Dans `Binaries\NCP_Binaries`, copier `mission_t01_flash_prog_cfg.ini` en `cfg_obcp.ini`, puis, dans la section `[FW]` de la copie, remplacer `st67w611m_mission_t01_vxxx.bin` par le nom du binaire téléchargé :

```
[FW]
filedir = ./st67w611m_mission_t01_v2.0.106.bin
address = @partition
```

Étape 3 – Programmer le module

1. Dans le terminal de l'ObCP, taper `module boot` : le module redémarre en mode téléchargement (BOOT = 1 pendant le redémarrage, comme avec l'utilitaire NCP_Loader de ST), la LED RVB passe au magenta et la procédure s'affiche (figure ci-dessous).
2. **Fermer le terminal** : le port COM doit être libre pour QConn_Flash.
3. Ouvrir une invite de commandes dans le dossier `Binaries` et lancer QConn_Flash sur le port de l'ObCP (COM7 dans l'exemple ; sous Linux, deuxième forme) :

```
QConn_Flash\QConn_Flash_Cmd.exe --port COM7 ^
    --config NCP_Binaries\cfg_obcp.ini --efuse=NCP_Binaries\efusedata.bin

# Linux
QConn_Flash/QConn_Flash_Cmd-ubuntu --port /dev/ttyACM0 \
    --config NCP_Binaries/cfg_obcp.ini --efuse=NCP_Binaries/efusedata.bin
```

4. QConn_Flash efface la flash du module, la programme (chargeur boot2, table des partitions, firmware, système de fichiers) et vérifie l'écriture ; compter une à deux minutes. Comme le script de ST, la commande écrit les efuses du module (`efusedata.bin`) : le module n'accepte ensuite que des binaires signés par ST, ce qui est le cas de tous les firmwares publiés.
5. Quand QConn_Flash a terminé, **rouvrir le terminal sans rien taper** : après 2 s de silence, la passerelle s'arrête, affiche un bilan (octets échangés, durée), puis le démonstrateur redémarre le module et affiche sa version : « Firmware : 2.0.106 ».



Figure 4 – Première programmation du module : consignes de module boot et bilan de la passerelle

La passerelle s'arrête aussi par un appui sur le bouton **User1** (ObCP) ou **SW1** (X-NUCLEO), ou d'elle-même après 2 minutes sans échange (15 minutes si QConn_Flash n'a jamais été lancé). Elle ne se fie pas au seul signal DTR du port COM : QConn_Flash le manipule pour tenter de redémarrer la puce. Les mises à jour suivantes se font avec `module ota` (3.3).

> **Attention :** cette méthode n'a pas encore été essayée sur une ObCP réelle (voir 1.5). En cas d'échec de QConn_Flash : appuyer sur User1 pour quitter la passerelle, vérifier que le terminal est bien fermé et que le port est le bon, puis recommencer par `module boot`. Le chargeur en ROM du module ne s'efface pas : une programmation interrompue peut toujours être refaite.
>
> Solution de repli : la méthode 3.5, avec une Nucleo prise en charge par les scripts de ST.

3.5 Première programmation avec une NUCLEO-H563ZI

Les scripts de ST savent programmer le module enfiché sur une NUCLEO-U575ZI-Q, NUCLEO-H563ZI, NUCLEO-H7S3L8 ou NUCLEO-N657X0-Q. L'exemple ci-dessous utilise une **NUCLEO-H563ZI** sous Windows ; avec les autres cartes, seul le dossier `NUCLEO-…_Binaries` change.

1. Installer **STM32CubeProgrammer** dans son dossier par défaut (`C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin`) : le script de ST l'y cherche.

2. Récupérer les fichiers comme en 3.4, étape 1, en ajoutant `Binaries\NCP_update_mission_profile_t01.bat` et le dossier `Binaries\NUCLEO-H563ZI_Binaries` (`Bootloader.bin`, `ST67W6X_CLI.bin`). Le fichier `cfg_obcp.ini` n'est pas nécessaire : le script prépare lui-même sa configuration.
3. Retirer la X-NUCLEO-67W61M1 de l'ObCP et l'enficher sur les connecteurs Arduino (ZIO) de la NUCLEO-H563ZI, cavaliers JP1 et JP2 en place. Relier **uniquement** la NUCLEO-H563ZI au PC par l'USB de son ST-LINK et fermer tout terminal série.
4. Double-cliquer sur `Binaries\NCP_update_mission_profile_t01.bat`, répondre **Y** à l'avertissement « You are about to load a signed binary… This will lock the ST67W61M » (normal : le module n'accepte de toute façon que des binaires signés par ST). Le script détecte la carte, charge `Bootloader.bin` dans la H563ZI, programme le module (environ une minute), puis charge l'application de test `ST67W6X_CLI.bin`.
5. Remettre la X-NUCLEO-67W61M1 sur l'ObCP : la bannière de démarrage doit indiquer « Firmware : 2.0.106 ».

> **Bon à savoir :** le programme présent dans la NUCLEO-H563ZI est remplacé par l'application de test de ST : le recharger ensuite si besoin. Le rôle des fichiers est décrit dans le README des scripts.

4. Programmation de l'ObCP et compilation

4.1 Le chargeur USB DFU

L'ObCP n'a pas de sonde ST-LINK : le programme se charge par l'USB, grâce au **chargeur DFU** (Device Firmware Upgrade) gravé en ROM dans le STM32 par ST. Quand il est actif, l'ObCP apparaît sur le PC comme un périphérique « STM32 BOOTLOADER » (et non plus comme un port COM), que STM32CubeProgrammer sait programmer. Deux façons de l'activer :

- **Interrupteur et bouton** (toujours possible, même carte vierge ou programme bloqué) : placer **SW4** sur **Prog** et appuyer sur **RST** (SW3). Après la programmation, remettre SW4 sur **Run** et appuyer sur RST.
- **Commande** `dfu` (quand le démonstrateur fonctionne) : taper `dfu` dans le terminal puis la touche **o** pour confirmer. L'ObCP redémarre dans le chargeur sans qu'il soit nécessaire de toucher à SW4 et RST, ce qui est pratique quand la X-NUCLEO les recouvre. Le port COM disparaît ; après la programmation, le nouveau programme démarre (« Run after programming ») ou au prochain reset.

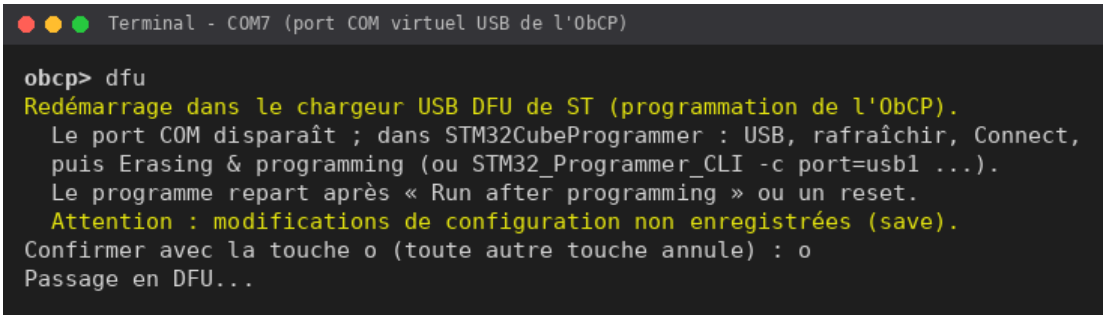


Figure 5 – Passage dans le chargeur USB DFU par la commande dfu

> **Bon à savoir :** le pilote USB du chargeur DFU est installé avec STM32CubeProgrammer sous Windows ; sous Linux, copier les règles udev fournies avec STM32CubeProgrammer (dossier `Drivers/rules`) dans `/etc/udev/rules.d`.

4.2 Programme précompilé avec STM32CubeProgrammer

Le dossier `Binaries/` contient le programme déjà compilé (équivalent de la configuration Release) :

1. Activer le chargeur DFU (4.1).
2. Dans STM32CubeProgrammer, choisir **USB** dans la liste des interfaces (à la place de ST-LINK), cliquer sur le bouton d'actualisation : le port **USB1** apparaît. Cliquer sur **Connect**.
3. Ouvrir l'onglet **Erasing & programming**, choisir `Binaries/ObCP_WiFi_Demo.hex` (ou le `.elf` ; avec le `.bin`, indiquer l'adresse de début 0x08000000), cocher **Verify programming** et **Run after programming**, puis **Start Programming**.
4. Si SW4 a été utilisé, le remettre sur **Run** et appuyer sur RST. Ouvrir ensuite le terminal (chapitre 5).

En ligne de commande, la même opération s'écrit :

```
STM32_Programmer_CLI -c port=usb1 -w Binaries/ObCP_WiFi_Demo.hex -v -s
```

> **Astuce :** la programmation n'efface que les pages couvertes par le fichier : la configuration enregistrée (réseau Wi-Fi, MQTT…), stockée dans la dernière page de la flash, est conservée. Un **Full chip erase** l'efface aussi.

4.3 Avec STM32CubeIDE

1. Décompresser le paquet, puis dans STM32CubeIDE : **File › Import › General › Existing Projects into Workspace**, choisir le dossier `ObCP_WiFi_Demo` (celui qui contient `.project`), puis **Finish**.

2. Choisir la configuration **Debug** (optimisation -Og) ou **Release** (-Os) et compiler (marteau). Les deux configurations compilent sans avertissement et produisent aussi les fichiers `.bin` et `.hex`.
3. Pour programmer l'ObCP depuis STM32CubeIDE, créer une fois un outil externe, comme dans le dépôt ObCP : **Run › External Tools › External Tools Configurations…**, nouvelle configuration de type Program avec les réglages ci-dessous.

Champ	Valeur
Location	`C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin\STM32_Programmer_CLI.exe`
Working Directory	`${workspace_loc}${project_path}\Debug` (ou `\Release`)
Arguments	`-c port=usb1 -d ${project_name}.elf -v -s`

4. Activer le chargeur DFU (`dfu` dans le terminal, ou SW4 sur Prog et RST), puis lancer l'outil externe : le programme est chargé, vérifié et démarré.

> **Bon à savoir :** la configuration du matériel est décrite par `ObCP_WiFi_Demo.ioc` : l'ouvrir pour la consulter ou la modifier (4.4). Le dossier `Tests/` (programmes pour PC) n'est pas compilé par CubeIDE : ce n'est pas un dossier source du projet.
>
> **Débogage pas à pas** : sans ST-LINK sur l'ObCP, **Run › Debug As** nécessite une sonde SWD externe reliée au connecteur Debug J17 : TCK = SWCLK, TMS = SWDIO, RST = NRST, GND ; la référence de tension de la sonde se prend sur la broche 3V3 du connecteur J6. Le ST-LINK d'une carte Nucleo-64 convient (cavaliers CN2 retirés, connecteur SWD CN4).

4.4 Modifier la configuration avec STM32CubeMX

Le projet est un projet STM32CubeMX ordinaire : `ObCP_WiFi_Demo.ioc` décrit les broches (avec leurs étiquettes), les horloges et les périphériques, et l'initialisation correspondante (`main.c`, `gpio.c`, `spi.c`, `adc.c`, `tim.c`, `rtc.c`, `USB_DEVICE/`…) est générée par CubeMX. On peut donc ajouter un capteur, une entrée analogique ou un bus dans CubeMX et régénérer le code : la démo continue de fonctionner, pour deux raisons.

- Le code de la démo est rangé dans des **fichiers que CubeMX ne génère pas** : `board.c`, `cdc_port.c`, `fault_handlers.c`, `uart_bridge.c`, `ncp_port_obcp.c`, les dossiers `Core/Src/app`, `Core/Src/st67w6x` et `Core/Src/qrcodegen`, leurs en-têtes dans `Core/Inc`. CubeMX ne les touche jamais.
- Il est relié au code généré par quelques lignes écrites **entre les commentaires** `/* USER CODE BEGIN … */` et `/* USER CODE END … */`, que CubeMX recopie à chaque génération (tableau ci-dessous).

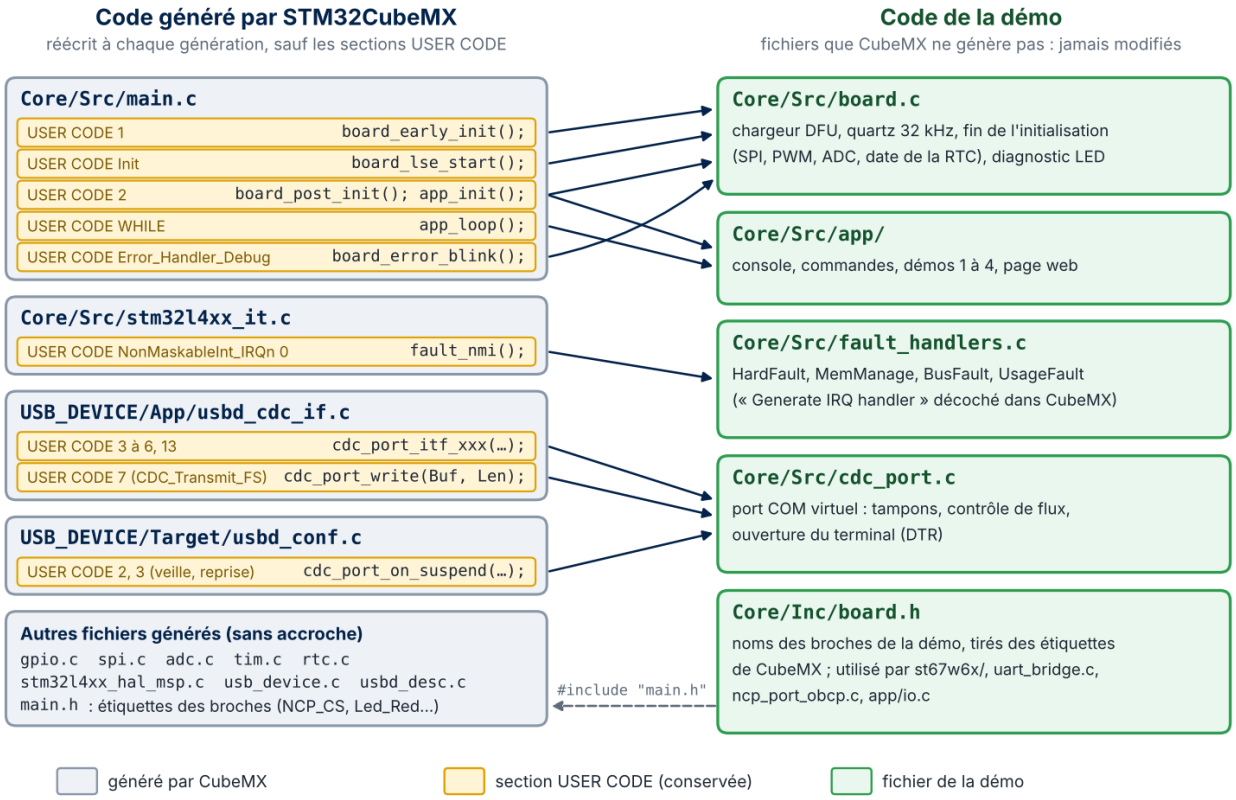


Figure 6 – Code généré par CubeMX et code de la démo : les liens passent par les sections USER CODE

Fichier généré	Section USER CODE	Contenu
`Core/Src/main.c`	`1`	`board_early_init();` : saut éventuel dans le chargeur DFU, avant toute initialisation
	`Init`	`board_lse_start();` : quartz 32 kHz démarré avec l'amplification « medium high »
	`2`	`board_post_init();` (SPI actifs, PWM démarrées, ADC étalonné, date de la RTC) puis `app_init();`
	`WHILE`	`app_loop();` dans la boucle principale
	`Error_Handler_Debug`	`board_error_blink();` : diagnostic par la LED rouge (chapitre 12)
`Core/Src/stm32l4xx_it.c`	`PFP`, `NonMaskableInt_IRQn 0`	`fault_nmi();` (erreur ECC de la configuration en flash, sinon rapport de faute)
`USB_DEVICE/App/usbd_cdc_if.c`	`INCLUDE`, `3`, `4`, `5`, `6`, `7`, `13`	rappels de la classe CDC transmis à `cdc_port.c` ; `CDC_Transmit_FS()` passe par son tampon
`USB_DEVICE/Target/usbd_conf.c`	`Includes`, `2`, `3`	`cdc_port_on_suspend(true / false);` : veille et reprise de l'USB

Régénérer avec STM32CubeIDE 1.x (éditeur de configuration intégré)

1. Double-cliquer sur `ObCP_WiFi_Demo.ioc`. À la première ouverture, CubeIDE peut proposer de migrer le projet vers sa version de CubeMX et vers le paquet STM32Cube FW_L4 installé : accepter.
2. Modifier la configuration (onglets **Pinout & Configuration** et **Clock Configuration**).
3. Enregistrer (**Ctrl+S**) et répondre **Yes** à « Do you want to generate Code? » (ou **Project › Generate Code**).

4. Vérifier le projet puis compiler (voir plus bas).

Régénérer avec STM32CubeMX (STM32CubeIDE 2.x)

Depuis STM32CubeIDE 2.0, CubeMX n'est plus intégré à l'IDE : le `.ioc` s'ouvre dans STM32CubeMX, installé à part.

1. Dans STM32CubeMX : **File › Load Project**, choisir `ObCP_WiFi_Demo.ioc` ; accepter la migration éventuelle.
2. Modifier la configuration. Dans **Project Manager**, la chaîne d'outils reste **STM32CubeIDE** (déjà réglée), avec « Generate peripheral initialization as a pair of .c/.h files » coché.
3. Cliquer sur **GENERATE CODE**.
4. Dans STM32CubeIDE, sélectionner le projet et l'actualiser (**F5**), puis vérifier et compiler.

Vérifier après chaque régénération

Le script `Tools/verif_cubemx.py` (Python 3) contrôle que les accroches sont toutes en place, que les étiquettes des broches et les périphériques dont la démo a besoin existent toujours et que les gestionnaires de faute ne sont pas générés :

```
python Tools/verif_cubemx.py            (dans le dossier du projet)
Points d'accroche dans les sections USER CODE : 20/20 présents
Périphériques, étiquettes des broches et gestionnaires de faute : conformes
Projet prêt à compiler.
```

En cas de problème, il indique le fichier et la section concernés. La même vérification est faite par `make -C Tests check-cubemx`.

Règles à respecter

- Dans les fichiers générés, n'écrire **qu'entre** `USER CODE BEGIN` et `USER CODE END` : tout ce qui est écrit ailleurs est effacé à la régénération. Pour du code nouveau, créer ses propres fichiers dans `Core/Src` et `Core/Inc` : ils sont compilés sans autre réglage.
- Garder les **étiquettes** des broches (`NCP_CS`, `NCP_RDY`, `NCP_CHIP_EN`, `NCP_BOOT`, `NCP_SW1`, `NCP_UART_TX`, `NCP_UART_RX`, `Led_Red`, `Led_Green`, `Led_Blue`, `PWM_Motor`, `ACC_CS`, `User1`, `BLE_Reset`) : `Core/Inc/board.h` s'appuie sur elles. Déplacer une broche en gardant son étiquette suffit.
- Laisser décochée la case **Generate IRQ handler** des lignes « Hard fault interrupt », « Memory management fault », « Prefetch fault, memory access fault » et « Undefined instruction or illegal state » (**System Core › NVIC › Code generation**) : ces gestionnaires sont dans `fault_handlers.c`, écrits en assembleur pour relever l'adresse de la faute.
- Ne pas désactiver SPI1, SPI3, ADC1, TIM1, TIM3, la RTC ni l'USB (USB_OTG_FS et USB_DEVICE en classe CDC). Ne pas activer l'**USART2** : la passerelle `module boot` le pilote elle-même, au niveau des registres et seulement pendant la session ; PA2 et PA3 restent des entrées le reste du temps.
- Les réglages de CubeMX restent maîtres : la démo lit la période des PWM dans les timers, choisit le canal de l'ADC à chaque mesure et ne fixe pas le débit des SPI ; un réglage modifié dans CubeMX est donc pris en compte tel quel.

> **Bon à savoir :** aucun chemin d'inclusion ni dossier source n'a été ajouté à la main au projet CubeIDE : tout le code de la démo est sous `Core/`, que CubeMX déclare lui-même. La compilation reste donc possible même si une régénération réécrit les réglages du projet ; seules les options choisies ici (optimisation -Og en Debug, symbole `NDEBUG` en Release) reviendraient alors aux valeurs par défaut, sans effet sur le fonctionnement.

> **Attention :** différence avec la version 1.0 : l'initialisation générée par CubeMX ne prévoit pas de repli automatique sur les oscillateurs internes. Si le quartz 8 MHz ou le quartz 32,768 kHz ne démarre pas, le programme s'arrête dans `Error_Handler()` et la LED rouge clignote : 2 éclats pour le quartz 8 MHz, 3 éclats (au bout d'une dizaine de secondes) pour le quartz 32 kHz. Une carte dont le quartz 32 kHz est défectueux peut fonctionner avec la RTC sur l'oscillateur interne LSI (chapitre 12).

Exemple : ajouter une entrée analogique sur A0

1. Dans CubeMX, **Analog › ADC1** : cocher **IN5 Single-ended** (broche PA0, A0 du connecteur Arduino), puis donner l'étiquette `POT` à PA0 (clic droit › Enter User Label).
2. Régénérer, vérifier avec `Tools/verif_cubemx.py`.
3. Dans `Core/Src/app/io.c`, lire la tension avec la fonction de la démo : `adc_read(ADC_CHANNEL_5, ADC_SAMPLETIME_92CYCLES_5)` (moyenne de 16 conversions, 0 à 4095 pour 0 à VDDA), puis l'ajouter par exemple à `io_read()`.

4.5 En ligne de commande

Avec la chaîne GNU Arm (`arm-none-eabi-gcc`) dans le PATH :

```
make            # build/ObCP_WiFi_Demo.elf, .hex et .bin
make flash      # programmation par USB DFU (STM32_Programmer_CLI, chargeur DFU actif)
make test       # tests sur PC (gcc natif)
make clean
```

4.6 Ressources utilisées

Ressource	Utilisation
Flash	≈ 137 Ko sur 1 Mo en Release (≈ 153 Ko en Debug) ; la dernière page de 2 Ko, à 0x080FF800, stocke la configuration
RAM	≈ 44 Ko sur les 96 Ko de SRAM1 (tampons USB, SPI, flux AT, console, serveur web, mise à jour du module, passerelle), plus 5 Ko réservés à la pile (4 Ko) et au tas (1 Ko), soit 49 Ko ; SRAM2 (32 Ko) reste libre
Horloges	Quartz 8 MHz → PLL 80 MHz (système) et PLLSAI1 48 MHz (USB) ; RTC sur le quartz 32,768 kHz (LSE, amplification « medium high »). Réglages dans CubeMX (Clock Configuration)
Périphériques	USB OTG FS (port COM virtuel), SPI1 (module), SPI3 (accéléromètre), ADC1 (température, VREFINT ; 16 conversions moyennées), TIM3 (LED RVB), TIM1 (sortie PWM), RTC ; USART2 et DMA1 seulement pendant `module boot`, hors CubeMX
Interruptions	USB OTG FS (priorité 2), SysTick (priorité 15) ; en cas de faute, rapport conservé dans les registres de sauvegarde de la RTC

5. Prise en main du terminal

5.1 Le port COM virtuel de l'ObCP

Le programme fait de l'ObCP un périphérique USB de la classe CDC : le PC voit un port série sans pilote à installer.

- **Windows 10 et 11** : « Périphérique série USB (COMx) » dans le Gestionnaire de périphériques, rubrique **Ports (COM et LPT)**. Le numéro (COM7 dans les exemples de ce guide) reste le même d'une connexion à l'autre sur le même port USB du PC.
- **Linux** : `/dev/ttyACM0` (l'utilisateur doit appartenir au groupe `dialout`) ; **macOS** : `/dev/cu.usbmodem…`.
- Le port apparaît une à deux secondes après la mise sous tension ou un reset et **disparaît pendant un reset**, un `reboot`, un passage en DFU ou une coupure : rouvrir alors la connexion du terminal (PuTTY : clic droit sur la barre de titre › Restart Session ; Tera Term : Fichier › Nouvelle connexion).
- Les messages produits avant l'ouverture du terminal (bannière de démarrage, évènements) sont conservés dans un tampon de 4 Ko et s'affichent dès l'ouverture. S'il déborde pendant que le terminal est fermé, le nombre d'octets perdus est indiqué ; à chaque réouverture, une ligne rappelle la carte et son identifiant :

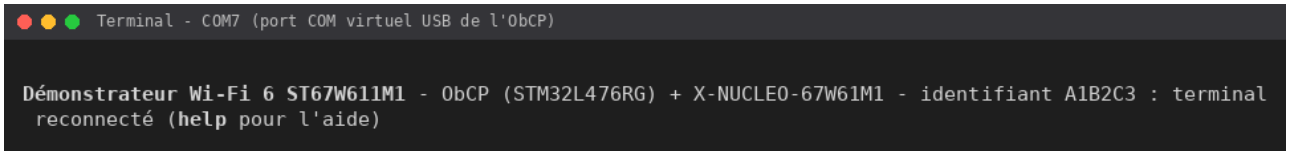


Figure 7 – Réouverture du terminal

5.2 Réglages du terminal

Paramètre	Valeur
Port	« Périphérique série USB (COMx) » (Windows), /dev/ttyACM0 (Linux)
Débit	Indifférent (115200 par exemple) : il n'est utilisé que par `module boot`
Format	8 bits, sans parité, 1 bit de stop, sans contrôle de flux
Jeu de caractères	UTF-8 (accents, barres de niveau, QR code)
Touche Entrée	envoie CR (réglage par défaut de PuTTY et Tera Term) ; écho local désactivé
Taille de fenêtre	au moins 100 colonnes (110 pour `scan bssid`)

- **PuTTY** : Connection type Serial ; Window › Translation : UTF-8.
- **Tera Term** : Configuration › Port série (Setup › Serial port en anglais) pour le port ; Configuration › Terminal (Setup › Terminal) : encodage UTF-8 en réception et en émission. Tera Term sait aussi envoyer un fichier en YMODEM (mise à jour du module, 3.3).
- **Linux / macOS** : `picocom /dev/ttyACM0` (quitter par Ctrl-A Ctrl-X).
- **Console de STM32CubeIDE** : elle ne gère pas les couleurs ANSI : taper `set term.color 0` puis `save`.

5.3 Premier démarrage

À l'ouverture du terminal (ou après un appui sur RST), la bannière rappelle la version, l'identifiant de la carte (6 caractères dérivés de l'identifiant unique du STM32, utilisés pour nommer le point d'accès et les topics MQTT),

l'état des quartz et de l'accéléromètre, la configuration chargée et les services qui démarrent seuls (ligne « Démarrage », 5.6), puis la version du firmware du module :

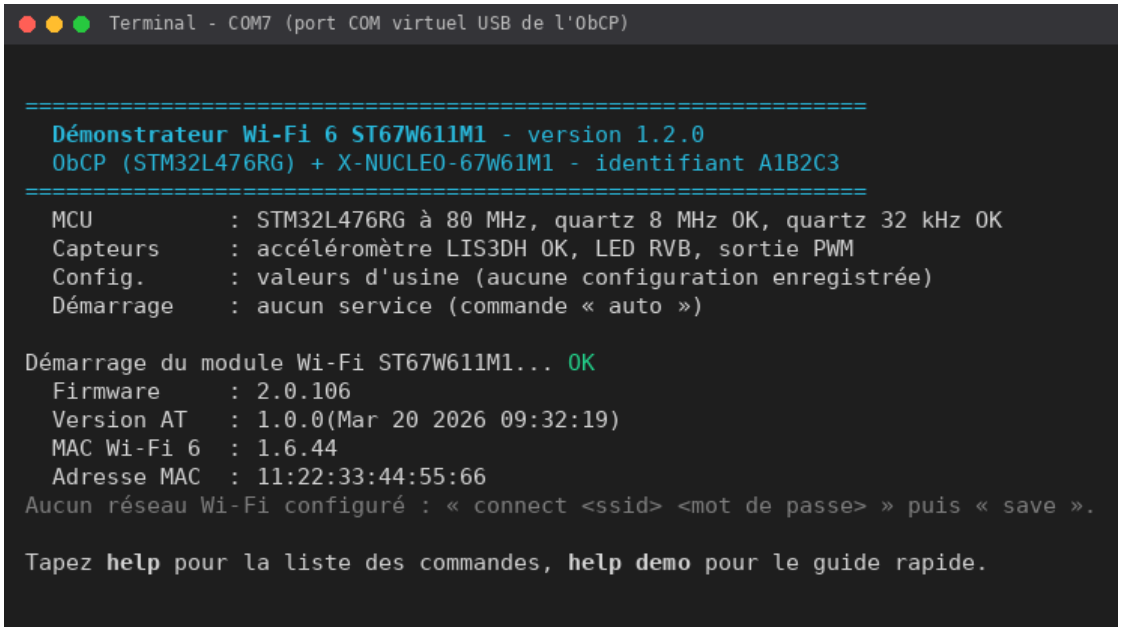


Figure 8 – Bannière de démarrage

- Si un quartz ne démarre pas, la bannière n'apparaît pas : le programme s'arrête pendant l'initialisation et la LED rouge clignote par séries de 2 éclats (quartz 8 MHz) ou de 3 éclats (quartz 32 kHz), voir le chapitre 12.
- « quartz 32 kHz non utilisé (RTC sur LSI, moins précise) » : la RTC a été passée sur l'oscillateur interne dans CubeMX (dépannage d'une carte dont le quartz 32 kHz ne démarre pas, chapitre 12).
- « accéléromètre absent ou non reconnu (WHO_AM_I = 0x…) » : le capteur ne répond pas (voir le dépannage, chapitre 12) ; le reste du démonstrateur fonctionne.
- « Redémarrage après une faute … » : le programme s'est arrêté sur une erreur grave lors de la session précédente (la LED a clignoté en rouge pendant 2 s avant le redémarrage) ; l'adresse fautive (PC) aide à retrouver la fonction en cause.

La LED bleue LD1 de la X-NUCLEO s'allume brièvement à chaque échange SPI (signal RDY) : c'est normal.

5.4 Utilisation de la ligne de commande

- `help` liste les commandes par démo (liste complète au chapitre 11), `help <commande>` donne le détail et des exemples, `help demo` affiche le guide rapide (figure ci-dessous).
- **Flèche haut** : rappel de la commande précédente ; **Ctrl-C** : annule la saisie ; **Retour arrière** : efface (accents compris).
- Un argument contenant des espaces se met entre guillemets : `connect "Ma Box" motdepasse`. Le caractère " est interdit dans les SSID et les mots de passe.
- Les évènements de fond (connexion perdue, client du point d'accès, message MQTT…) s'affichent au-dessus de la ligne en cours de saisie, sans la perdre.
- Les commandes longues (`track`, `sensors suivi`) s'arrêtent par l'appui sur une touche.

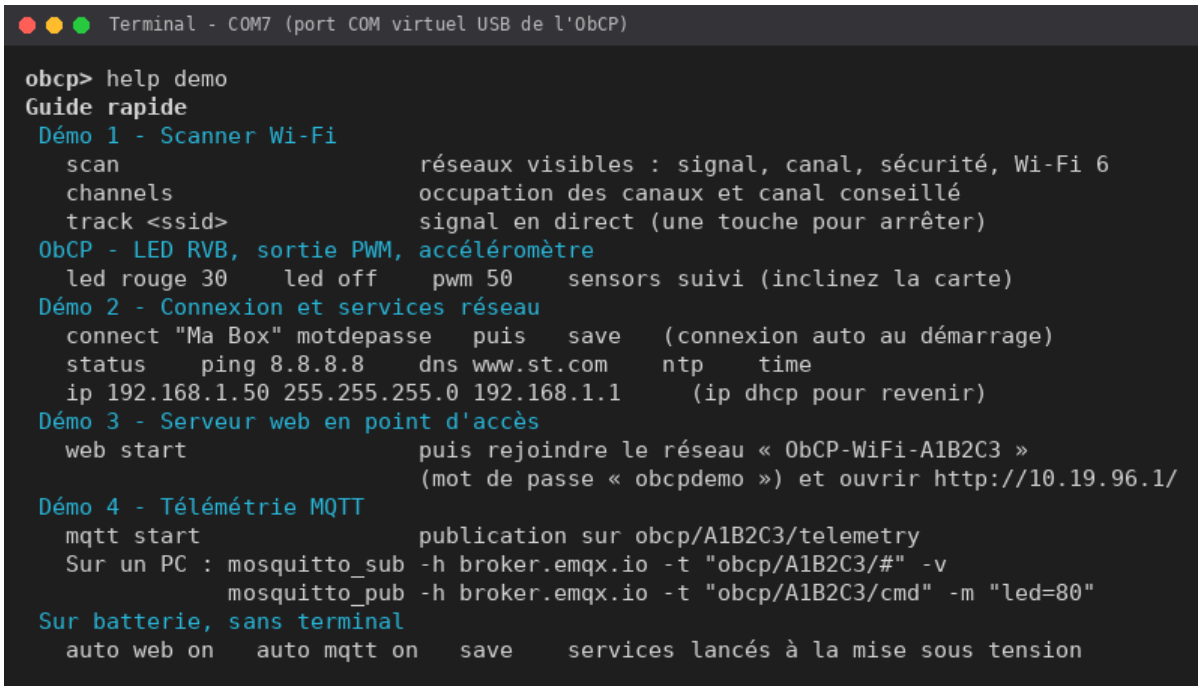


Figure 9 – Guide rapide (help demo)

5.5 Configuration enregistrée

Les réglages se consultent avec `config` (ou `config wifi`, `config mqtt`…), se modifient avec `set <clé> <valeur>` et sont enregistrés en flash par `save` ; `factory` rétablit les valeurs d'usine (`factory erase` efface aussi la flash). Maintenir **User1** appuyé pendant un reset démarre avec les valeurs d'usine sans toucher à la flash. Les mots de passe sont masqués à l'affichage. Après chaque `set`, le terminal indique quand le réglage prend effet (immédiatement, à la prochaine connexion, au prochain `web start`…). La commande `auto` résume et règle ce qui démarre seul à la mise sous tension (5.6).

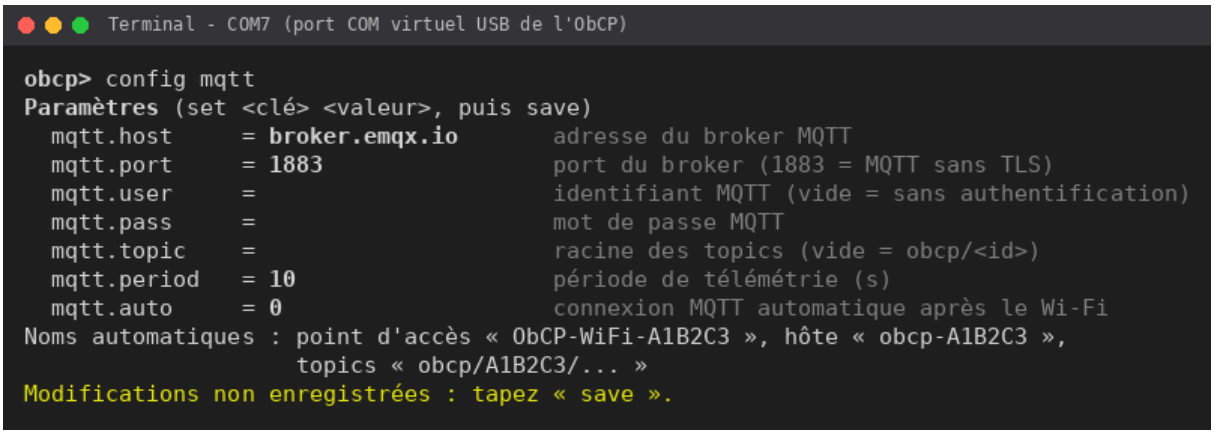


Figure 10 – Paramètres MQTT (config mqtt) et noms automatiques dérivés de l'identifiant

Clé	Défaut	Rôle
`wifi.ssid` / `wifi.pass`	(vide)	Réseau de la station (rempli aussi par `connect`)
`wifi.auto`	1	Connexion au démarrage et reconnexion automatique (5.6)
`wifi.country`	EU	Pays (canaux autorisés) : EU, US, JP, CN ou 00
`ip.dhcp`	1	1 = DHCP ; 0 = adresse fixe `ip.addr`, `ip.mask`, `ip.gw`, `ip.dns`
`net.hostname`	(vide) → obcp-<id>	Nom d'hôte annoncé au serveur DHCP

Clé	Défaut	Rôle
`time.server`	pool.ntp.org	Serveur NTP
`time.tz` / `time.dst`	60 / 1	Fuseau en minutes (60 = Paris) ; heure d'été européenne automatique
`time.auto`	1	Synchronisation NTP à chaque connexion (puis toutes les 6 h)
`ap.ssid` / `ap.pass`	(vide) → ObCP-WiFi-<id> / obcpdemo	Point d'accès de la démo 3 (WPA2 : 8 caractères minimum ; mot de passe vide = réseau ouvert)
`ap.channel` / `ap.ip`	6 / 10.19.96.1	Canal (1 à 13) et adresse du point d'accès
`web.port`	80	Port du serveur web
`web.auto`	0	Point d'accès et serveur web lancés à la mise sous tension (5.6)
`mqtt.host` / `mqtt.port`	broker.emqx.io / 1883	Broker MQTT (sans TLS)
`mqtt.user` / `mqtt.pass`	(vide)	Identifiants MQTT (vide = sans authentification)
`mqtt.topic`	(vide) → obcp/<id>	Racine des topics
`mqtt.period` / `mqtt.auto`	10 / 0	Période de télémétrie (2 à 3600 s) ; connexion MQTT automatique après le Wi-Fi (5.6)
`scan.period`	1000	Période du suivi RSSI (200 à 60000 ms)
`term.color`	1	Couleurs ANSI dans le terminal

5.6 Démarrage automatique (utilisation sur batterie)

Sur batterie, l'ObCP fonctionne sans PC ni terminal : ce qui doit tourner doit démarrer seul. Trois réglages enregistrés en flash décident de ce qui se lance à la mise sous tension ; la commande `auto` les affiche et les modifie :

Réglage	Commande	Défaut	Ce qui démarre seul
`wifi.auto`	`auto wifi on`	1	Connexion à la box enregistrée (par `connect` puis `save`), puis reconnexion après une coupure (8.4)
`web.auto`	`auto web on`	0	Point d'accès ObCP-WiFi-<id> et serveur web (démo 3), une fois la tentative de connexion à la box terminée
`mqtt.auto`	`auto mqtt on`	0	Télémétrie MQTT (démo 4), une seconde après que la carte a obtenu son adresse sur la box : il faut une box enregistrée, avec accès à Internet

1. Avec le terminal, connecter une fois la carte à la box : `connect "Livebox-7A3C" monmotdepasse` (8.1).
2. Choisir les services : `auto web on` et/ou `auto mqtt on` (`off` pour les retirer), puis `save`. `auto` seul affiche le résumé sans rien changer.
3. Alimenter la carte par sa batterie (connecteur J1, interrupteur SW2 sur ON) : à chaque mise sous tension, ou après un appui sur RST, elle rejoint la box, ouvre son point d'accès et publie sa télémétrie, sans terminal.

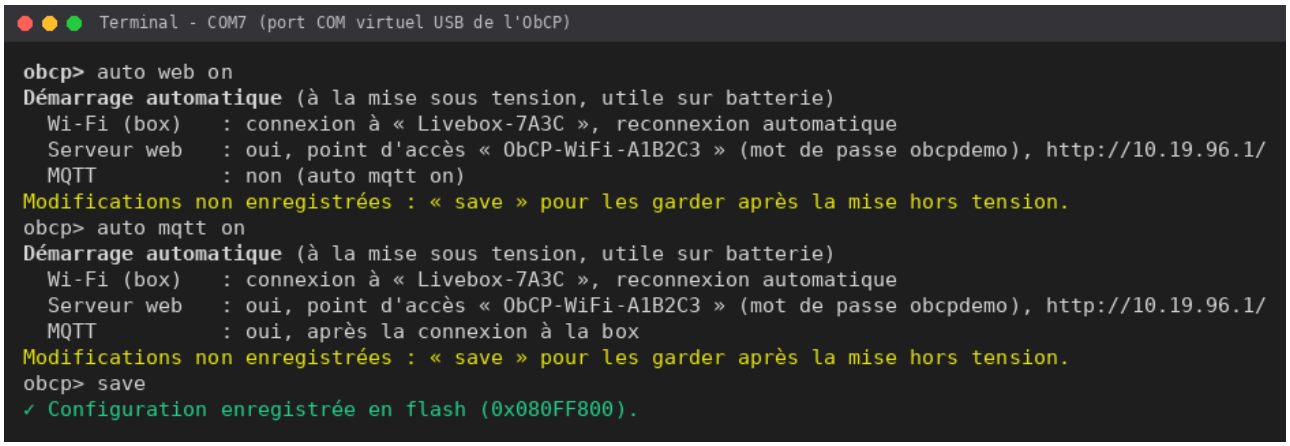


Figure 11 – Réglage du démarrage automatique (commande auto)

La bannière rappelle sur sa ligne « Démarrage » ce qui va se lancer ; avec un terminal ouvert, chaque étape s'affiche ensuite :

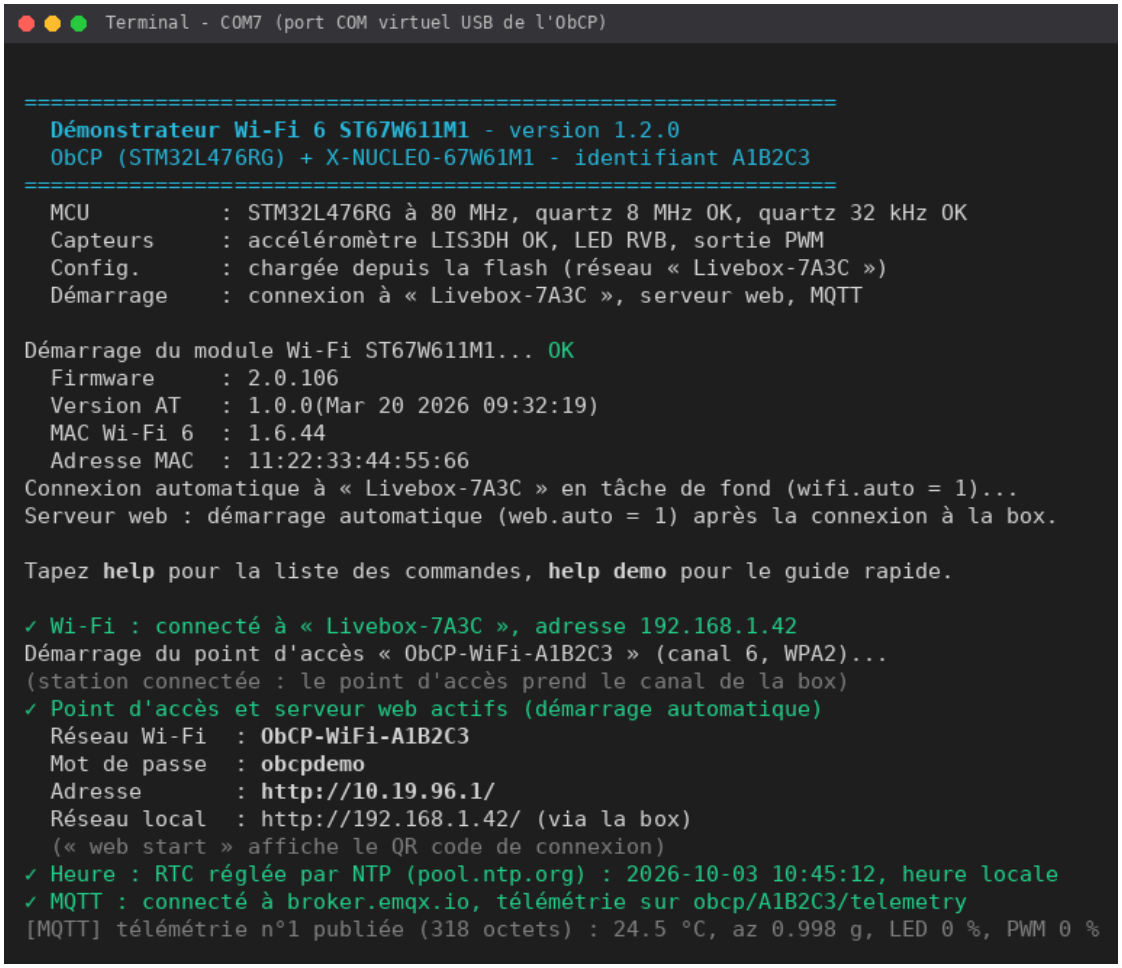


Figure 12 – Mise sous tension avec la box, le serveur web et MQTT en démarrage automatique

- Le point d'accès démarre une fois la tentative de connexion à la box terminée (35 s d'attente au plus), directement sur le canal de la box (une seule radio, 9.3). Si la box est absente ou refuse la connexion, il démarre dès l'échec et la carte continue d'essayer de rejoindre la box en tâche de fond (8.4) ; sans box enregistrée, il démarre aussitôt.
- En démarrage automatique, le QR code n'est pas affiché (il n'y a peut-être aucun terminal ouvert) : `web start` l'affiche à la demande. `web stop` arrête le serveur, ou annule un démarrage en attente, pour la session en cours seulement.

- Si le module redémarre de lui-même, le point d'accès et le serveur sont relancés ; MQTT se reconnecte seul après une coupure (10.2).
- La carte « Démarrage automatique » de la page web (9.2) fait le même réglage depuis le téléphone, et l'enregistre aussitôt en flash.
- Pour reprendre la main sur une carte ainsi réglée : `auto web off`, `auto mqtt off`, puis `save` ; ou maintenir User1 appuyé pendant un reset, ce qui démarre avec les valeurs d'usine, sans aucun service automatique (5.5).

> **Attention :** le point d'accès reste actif tant que la carte est alimentée : avant de laisser une carte en démarrage automatique sans surveillance, remplacer le mot de passe par défaut (`set ap.pass <mot de passe>` puis `save`, 9.3).

6. LED RVB, sortie PWM et accéléromètre

Ces commandes pilotent directement les composants de l'ObCP décrits en 2.5 ; la page web (démo 3) et les commandes MQTT (démo 4) agissent sur les mêmes réglages.

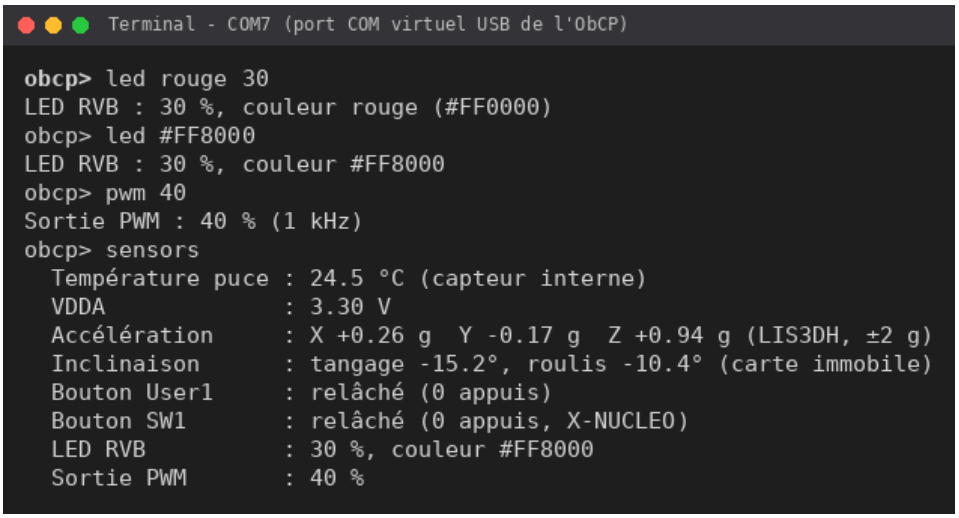


Figure 13 – Couleur de la LED RVB, sortie PWM et lecture des capteurs

6.1 LED RVB

```
led                 état actuel (luminosité, couleur)
led 50              luminosité 50 % (couleur inchangée)
led rouge           couleur (allume la LED si elle était éteinte)
led bleu 20         couleur et luminosité
led #FF8000         couleur en hexadécimal (ou led 255,128,0)
led off, led on     extinction, rallumage au dernier niveau
```

Couleurs nommées : rouge, vert, bleu, jaune, cyan, magenta, blanc, orange, violet, rose (les noms anglais red, green, blue… sont aussi acceptés). Le bouton **User1** allume ou éteint la LED. La LED sert aussi d'indicateur : du rouge au vert selon le signal pendant `track` (démo 1), bleue pendant `module ota`, magenta pendant `module boot`, rouge clignotante en cas de faute du programme.

6.2 Sortie PWM

`pwm 0` à `pwm 100` règle le rapport cyclique de la sortie du bornier J13 (fréquence 1 kHz) ; `pwm on` correspond à 100 %, `pwm off` à 0 %. Avec un petit moteur à courant continu, la vitesse suit le rapport cyclique ; avec une bande de LED, la luminosité. La charge se branche entre les bornes +5 V et PWM (2.5) ; ne pas dépasser 2,5 W.

6.3 Accéléromètre et capteurs

`sensors` affiche la température de la puce (capteur interne du STM32, étalonné en usine), la tension VDDA, les accélérations sur les trois axes (en g, 1 g = pesanteur), les inclinaisons, l'état des boutons et des sorties. `sensors suivi` rafraîchit ces valeurs sur une seule ligne jusqu'à l'appui d'une touche : incliner la carte pour voir varier les angles.

- **Tangage** : rotation autour de l'axe Y (le bord du connecteur Debug monte ou descend) ; **roulis** : rotation autour de l'axe X (le bord des connecteurs D0 à D15 monte ou descend). Les angles sont calculés à partir de la pesanteur : ils n'ont de sens que si la carte est immobile.
- Les mêmes valeurs alimentent la page web (bille qui roule vers le côté le plus bas) et la télémétrie MQTT (champs `ax`, `ay`, `az`, `pitch`, `roll`).

7. Démo 1 – Scanner Wi-Fi et suivi du signal

Objectifs : lire un niveau de signal en dBm, comprendre les canaux de la bande 2,4 GHz et leur recouvrement, reconnaître les types de sécurité et repérer les réseaux Wi-Fi 6 (802.11ax).

7.1 Balayage des réseaux

La commande `scan` lance un balayage actif des canaux 1 à 13 (2 à 3 s) et affiche les réseaux triés par niveau de signal :

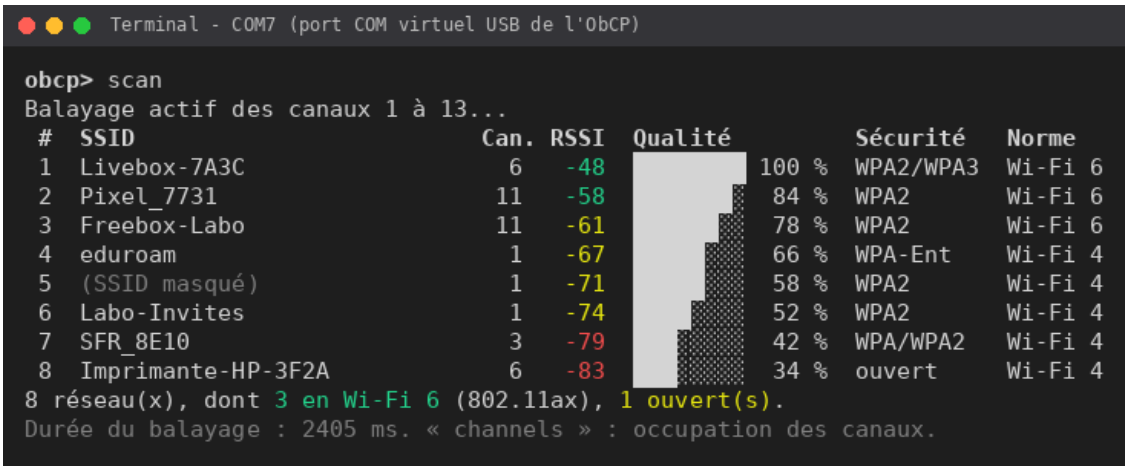


Figure 14 – Résultat de la commande scan

- **RSSI** : puissance reçue en dBm (vert ≥ -60 dBm, jaune ≥ -75 dBm, rouge en dessous). La colonne Qualité la convertit en pourcentage (−100 dBm → 0 %, −50 dBm → 100 %).
- **Sécurité** : ouvert, WEP, WPA, WPA2, WPA3 ou mixte ; WPA-Ent (entreprise, comme eduroam) n'est pas accepté par `connect`.
- **Norme** : Wi-Fi 6 (802.11ax) ou Wi-Fi 4 (802.11n), d'après les capacités annoncées par le point d'accès.
- Un réseau dont le nom n'est pas diffusé apparaît comme « (SSID masqué) ».
- Options : `scan passif` (écoute des balises seulement, plus lent), `scan canal 6` (un seul canal), `scan tri canal` ou `scan tri ssid`, `scan bssid` (ajoute l'adresse MAC de chaque point d'accès : plusieurs BSSID pour un même SSID révèlent un réseau maillé ou plusieurs bornes).

7.2 Occupation des canaux

`channels` calcule pour chaque canal un score d'occupation. Les canaux de la bande 2,4 GHz ne sont espacés que de 5 MHz alors qu'un réseau Wi-Fi occupe 20 MHz : chaque réseau compte donc à 100 % sur son canal et pour une part décroissante sur les quatre canaux voisins de chaque côté (70, 40, 15 puis 5 %), le tout pondéré par la puissance reçue. Le canal conseillé est le moins chargé parmi 1, 6 et 11, les seuls qui ne se chevauchent pas ; `channels appliquer` le reporte dans `ap.channel` pour le point d'accès de la démo 3 (`save` pour le garder). Le balayage est refait s'il date de plus d'une minute.

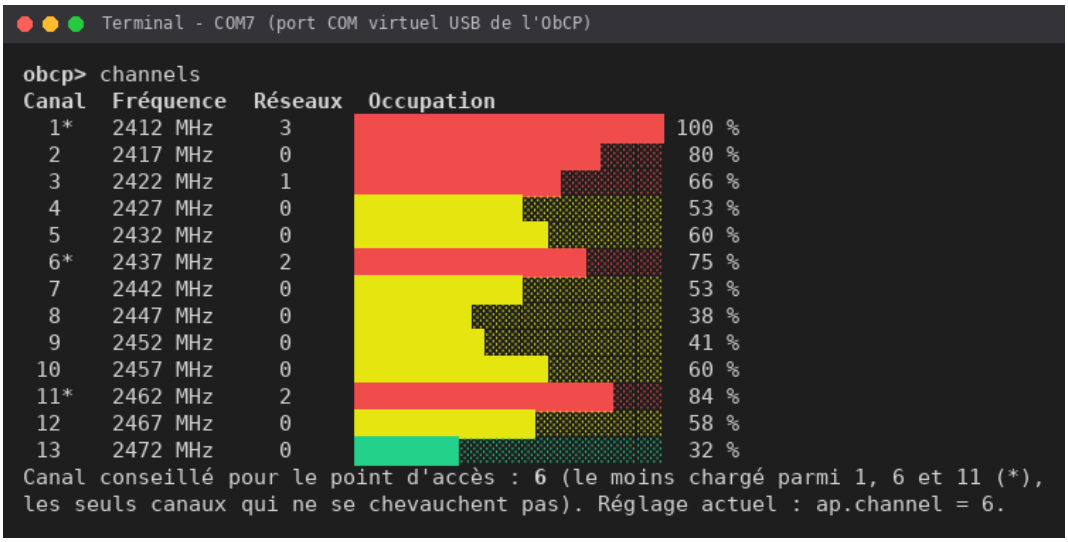


Figure 15 – Occupation des canaux et canal conseillé

7.3 Suivi du signal en direct

`track` mesure le RSSI toutes les `scan.period` ms : celui du réseau auquel la carte est connectée, ou celui d'un réseau donné (`track "Freebox-Labo"`, mesuré par balayage de son canal ; après cinq absences, tous les canaux sont balayés au cas où le point d'accès aurait changé de canal). Chaque mesure est horodatée (heure locale si la RTC est à l'heure). Pendant le suivi, la **LED RVB** passe du rouge (signal faible) au vert (signal fort) en passant par le jaune ; elle reprend ensuite sa couleur. Un appui sur une touche arrête la mesure et affiche le bilan (minimum, moyenne, maximum, écart-type, absences).

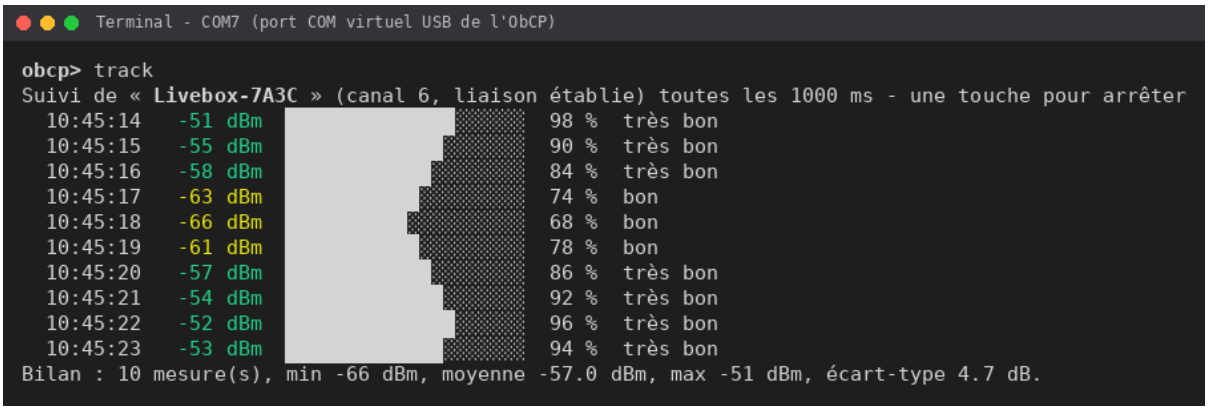


Figure 16 – Suivi du RSSI pendant un déplacement de la carte

Pour enregistrer une série et tracer la courbe, fermer le terminal puis lancer le script fourni (Python 3, modules pyserial et matplotlib) :

```
pip install pyserial matplotlib
python Tools/rssi_logger.py --port COM7 --duree 120 --sortie couloir.csv
```

Le script envoie `track csv` à la carte (`track "<ssid>" csv` avec l'option `--ssid`), qui répond par des lignes « temps_ms;rssi_dbm;canal ». Il enregistre les mesures dans un fichier CSV au format français (colonnes temps_s, rssi_dbm et canal, séparateur point-virgule, virgule décimale : le fichier s'ouvre directement dans un tableur), affiche le minimum, la moyenne et le maximum, puis trace la courbe dans un fichier PNG du même nom. L'option `--pas-de-graphe` supprime le tracé.

> **Astuce :** sur batterie, l'ObCP s'emporte partout et la télémétrie MQTT (démo 4) publie le RSSI sans terminal. Expériences simples : éloigner la carte de la box pièce par pièce, interposer la main, le corps ou une porte métallique, comparer deux orientations de la carte.

8. Démo 2 – Connexion et services réseau

8.1 Connexion à un réseau

```
connect "Livebox-7A3C" monmotdepasse
save
```

`connect` associe la carte au réseau, obtient une adresse par DHCP et affiche le détail de la liaison (point d'accès, canal, signal, sécurité, norme Wi-Fi, adresses). L'heure est synchronisée par NTP dans la foulée. Tant que `wifi.auto` vaut 1, la carte se reconnecte seule après une coupure ; `save` mémorise le réseau pour la connexion automatique au démarrage. `disconnect` coupe la liaison et suspend la reconnexion jusqu'au prochain `connect`.

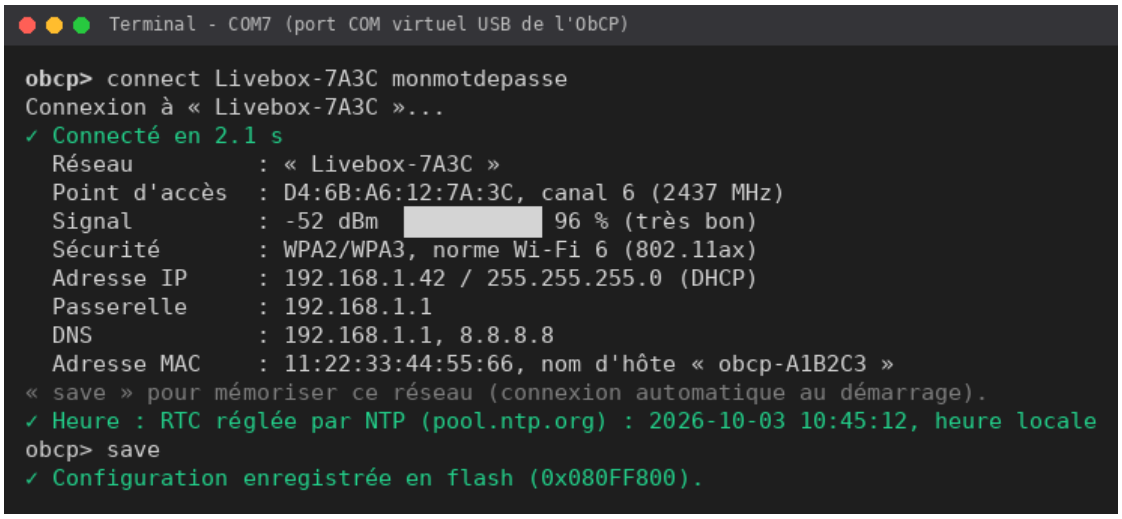


Figure 17 – Connexion, détail de la liaison et synchronisation NTP

En cas d'échec, la cause donnée par le module est traduite en clair et suivie d'un conseil :

Conseil affiché	Que faire
« Mot de passe incorrect ? »	Vérifier le mot de passe (majuscules, minuscules, caractères spéciaux)
« Réseau introuvable : vérifier le SSID »	Comparer avec le résultat de `scan` ; réseau 5 GHz seulement ; carte trop loin
« Sécurité non prise en charge par ce réseau »	Réseau WEP ou WPA-Entreprise : utiliser un autre réseau
« Pas de réponse DHCP »	Pas de serveur DHCP sur ce réseau : passer en adresse fixe (8.2)

> **Astuce :** sur un réseau d'établissement (eduroam, portail captif, ports filtrés), utiliser plutôt un routeur de TP ou le partage de connexion d'un téléphone : la carte ne gère ni WPA-Entreprise ni les portails captifs, et les ports UDP 123 (NTP) ou TCP 1883 (MQTT) peuvent être bloqués.

8.2 Adressage IP fixe ou DHCP

```
ip 192.168.1.50 255.255.255.0 192.168.1.1          (adresse, masque, passerelle)
ip 192.168.1.50 255.255.255.0 192.168.1.1 1.1.1.1  (avec un serveur DNS)
ip dhcp
```

`ip` seul affiche le mode actuel ; sans quatrième adresse, la passerelle sert de serveur DNS. Si la carte est connectée, la liaison est relancée avec le nouveau réglage (`save` pour le garder) ; un avertissement s'affiche si la passerelle n'est pas dans le sous-réseau de l'adresse. Comme le pilote officiel, le démonstrateur désactive le client DHCP du module, s'associe, puis applique l'adresse fixe (`AT+CIPSTA`) et le DNS (`AT+CIPDNS`).

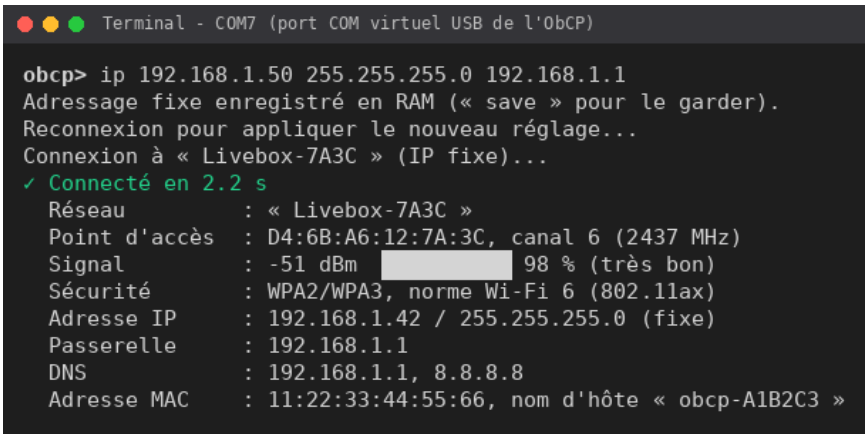


Figure 18 – Passage en adresse IP fixe

8.3 DNS, ping et heure

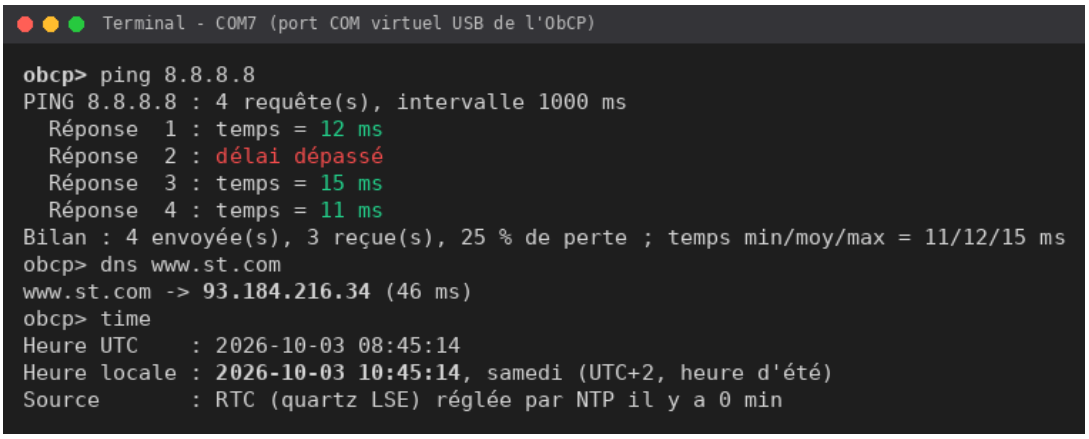


Figure 19 – Ping, résolution DNS et heure de la RTC

- `ping <hôte> [nombre] [intervalle]` : temps aller-retour de chaque écho, taux de perte et statistiques (4 échos par défaut, 1 à 50 ; intervalle de 100 à 3500 ms) ; un nom est d'abord résolu par DNS.
- `dns <nom>` : résolution par le serveur DNS du réseau, avec le temps de réponse.
- `ntp [serveur]` : interroge le serveur SNTP par le module et règle la **RTC du STM32 en UTC** ; à partir de la deuxième synchronisation, la correction appliquée (dérive de la RTC, à 1 s près) est affichée.
- `time` : heure UTC et heure locale. Le fuseau (`time.tz`, en minutes) et l'heure d'été européenne (`time.dst` : du dernier dimanche de mars au dernier dimanche d'octobre à 01:00 UTC) sont calculés par le STM32. La RTC, cadencée par le quartz 32,768 kHz de l'ObCP, continue de compter pendant un reset.

8.4 Ce qui se passe en tâche de fond

- **Reconnexion** : en cas de perte de liaison, première tentative au bout de 2 s puis, en cas d'échec, nouvel essai après 10, 20, 40 puis 60 s au plus, sans bloquer le terminal (`status` indique le délai restant).
- **NTP** : synchronisation à chaque connexion puis toutes les 6 heures (`time.auto`) ; sans réponse du serveur, nouvel essai 5 minutes plus tard.
- **Surveillance du module** : s'il redémarre de lui-même (message « ready »), le démonstrateur le réinitialise et rétablit la liaison.

9. Démo 3 – Serveur web en point d'accès

9.1 Démarrage

`web start` crée le réseau Wi-Fi de la carte (WPA2, ou ouvert si `ap.pass` est vide), sans couper la liaison de la station, et démarre le serveur HTTP du module sur le port `web.port` (80). Le terminal affiche le nom du réseau, son mot de passe, l'adresse à ouvrir et un **QR code** : l'appareil photo d'un téléphone le reconnaît et propose de rejoindre le réseau (QR code affiché seulement si `term.color` = 1).



Figure 20 – Démarrage du point d'accès et du serveur web, avec le QR code de connexion

1. Scanner le QR code (ou choisir le réseau ObCP-WiFi-<id> et saisir le mot de passe obcpdemo).
2. Accepter de rester connecté si le téléphone signale que le réseau n'a pas accès à Internet ; sur Android, désactiver au besoin les données mobiles, sinon le navigateur passe par le réseau mobile.
3. Ouvrir **http://10.19.96.1/** dans le navigateur. Si la carte est aussi connectée à une box, la page est accessible depuis le réseau local à l'adresse de la station.

Les arrivées et départs de clients s'affichent dans le terminal ; `web status` donne le nombre de requêtes et le dernier client, `web stop` arrête le serveur et le point d'accès.



Figure 21 – Client connecté au point d'accès et état du serveur web

Pour un usage sur batterie, sans terminal : `auto web on` puis `save`, et le point d'accès et le serveur démarrent seuls à chaque mise sous tension (5.6) ; la page web permet aussi ce réglage.

9.2 La page web

La page, stockée dans la flash du STM32 (environ 10 Ko, sans ressource externe : elle fonctionne sans accès à Internet), s'adapte au téléphone et au thème clair ou sombre de l'appareil. Elle interroge la carte toutes les 1,5 seconde :

- **Capteurs** : température de la puce, VDDA, accélérations, **bille** qui roule vers le côté le plus bas quand on incline l'ObCP (carte vue comme sur la figure de 2.2), tangage et roulis, état et compteurs des boutons, durée de fonctionnement, heure.
- **LED RVB et sortie PWM** : pastilles de couleur et sélecteur de couleur libre, curseur de luminosité, boutons Allumer et Éteindre, curseur de la sortie PWM ; le bouton User1 de la carte bascule aussi la LED.
- **Liaisons** : réseau de la station, RSSI, adresse IP, clients du point d'accès, état MQTT.
- **Réseaux voisins** : résultat du dernier balayage, bouton Actualiser (nouveau balayage de 3 s environ).
- **Connecter la carte à une box** : saisie du SSID (liste des réseaux proposée) et du mot de passe, enregistrement facultatif en flash. La connexion est lancée après l'envoi de la réponse.
- **Démarrage automatique** : cases à cocher pour lancer à la mise sous tension le point d'accès et le serveur web, et la télémétrie MQTT ; le bouton Enregistrer écrit aussitôt le réglage en flash (5.6).

Démonstrateur Wi-Fi 6 ST67W611M1

ObCP + X-NUCLEO-67W61M1 · identifiant A1B2C3 · version 1.2.0

Capteurs

Température de la puce	**24.5 °C**
Tension VDDA	**3.30 V**
Accélération X / Y / Z	**0.26 / -0.17 / 0.94 g**

Bille : roule vers le côté le plus bas. Tangage **-15°**, roulis **-10°**

Boutons User1 / SW1	**relâché (3) / relâché (1)**
En marche depuis	**1 h 02 min 12 s**
Heure (UTC)	**2026-10-26 09:41:07**

LED RVB et sortie PWM

Luminosité **100 % · #0000FF**

Allumer | Éteindre

Sortie PWM (bornier) **70 %**

Le bouton User1 de l'ObCP bascule aussi la LED.

Liaisons

Station (box)	**Livebox-7A3C**
Signal	**-52 dBm**
Adresse IP (box)	**192.168.1.42**
Clients du point d'accès	**1**
MQTT	**connecté**

Réseaux voisins — Actualiser

SSID	Canal	Signal	Sécurité	Norme
ObCP-WiFi-A1B2C3	6	-30 dBm	WPA2	Wi-Fi 6
Livebox-7A3C	6	-48 dBm	WPA2/WPA3	Wi-Fi 6
Freebox-Labo	11	-63 dBm	WPA2	Wi-Fi 6
eduroam	1	-71 dBm	WPA-Ent	Wi-Fi 4
(masqué)	1	-77 dBm	WPA2	Wi-Fi 4
Imprimante <HP>	6	-84 dBm	ouvert	Wi-Fi 4

Balayage effectué il y a 4 s.

Connecter la carte à une box

Nom du réseau (SSID)

Box Labo

Mot de passe

••••••••

☑ Enregistrer en flash (connexion au démarrage)

Se connecter — Connexion à « Box Labo » en cours… (suivi dans le terminal)

Le point d'accès reste actif. En se connectant à la box, le module peut changer de canal radio : ce téléphone peut alors être déconnecté quelques secondes.

Démarrage automatique

☑ Point d'accès et serveur web (cette page)
☑ Télémétrie MQTT, après la connexion à la box

Enregistrer — Enregistré : pris en compte au prochain démarrage.

Pris en compte à la prochaine mise sous tension, par exemple sur batterie et sans terminal : la carte rejoint alors la box enregistrée ci-dessus.

Figure 22 – Page web sur un téléphone : l'écran complet, présenté en deux colonnes (simulation)

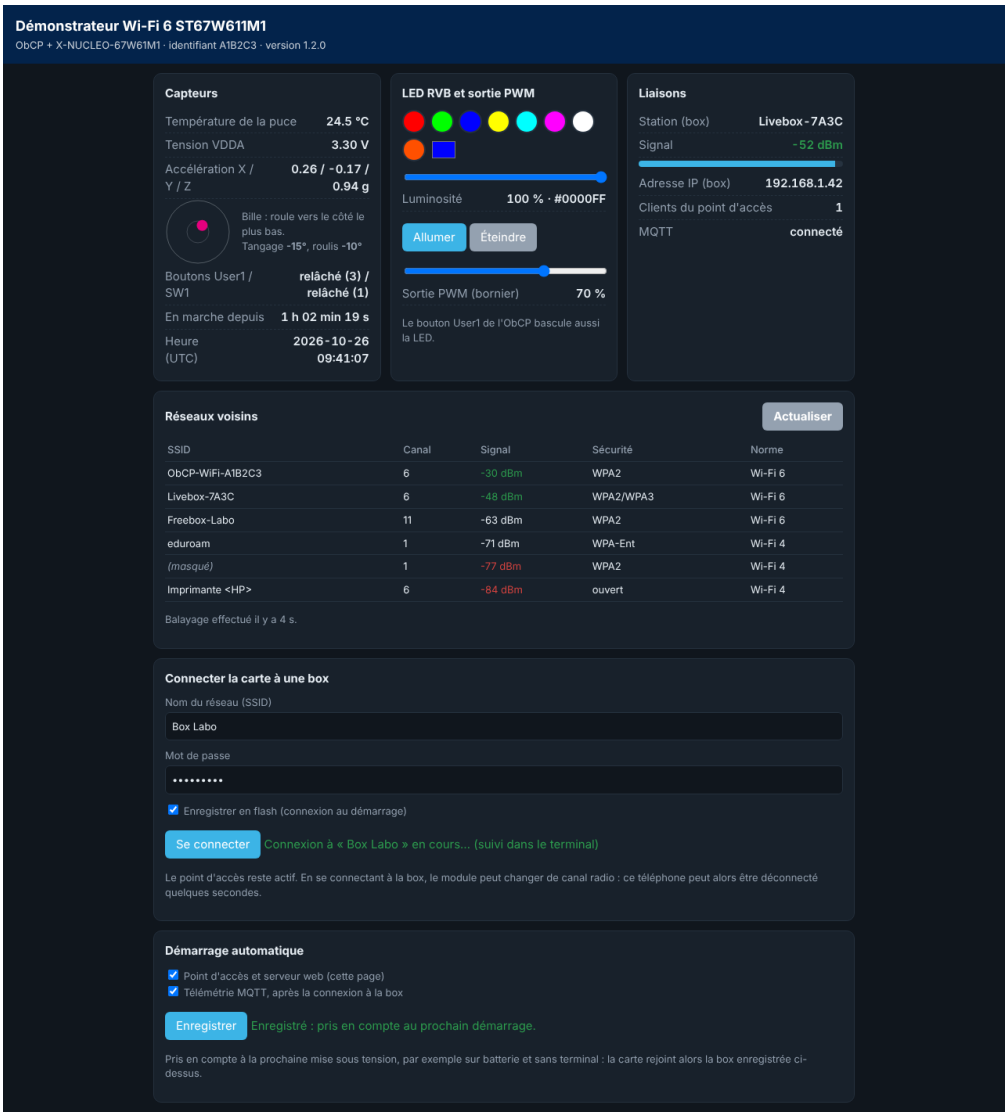


Figure 23 – Page web sur un ordinateur, thème sombre (simulation)

9.3 Interface HTTP

Les données de la page proviennent d'une petite API utilisable avec `curl` depuis un PC connecté au point d'accès :

Requête	Réponse
`GET /`	Page HTML du tableau de bord
`GET /api/status`	État de la carte en JSON (même contenu que la télémétrie MQTT, voir 10.3)
`GET /api/led?v=75&c=FF8000`	Règle la luminosité (`v`, 0 à 100) et/ou la couleur (`c` : RRGGBB ou nom) ; renvoie `{"led":75,"rgb":"#FF8000"}`
`GET /api/pwm?v=30`	Règle la sortie PWM ; renvoie `{"pwm":30}`
`GET /api/scan[?refresh=1]`	Réseaux voisins en JSON (avec `refresh=1` : nouveau balayage)
`POST /api/wifi`	Formulaire `ssid`, `pass`, `save=1` : enregistre le réseau et lance la connexion
`GET /api/auto[?web=1&mqtt=1]`	Démarrage automatique : renvoie `{"web":true,"mqtt":true,"saved":true}` ; avec `web` et/ou `mqtt` (0 ou 1), modifie `web.auto` et `mqtt.auto` et les enregistre en flash

```
curl http://10.19.96.1/api/status
curl "http://10.19.96.1/api/led?v=100&c=vert"
curl "http://10.19.96.1/api/pwm?v=50"
```

> **Attention :** démonstration pédagogique : HTTP sans chiffrement, page sans authentification (le mot de passe du point d'accès protège l'accès). Ne pas laisser le point d'accès actif sans surveillance avec le mot de passe par défaut : le modifier avec `set ap.pass`. Attention aussi à ce qui est branché sur la sortie PWM, commandable depuis la page.

Limites : quatre appareils connectés au plus, une requête traitée à la fois, connexion fermée après chaque réponse ; quand la station est connectée à une box, le point d'accès prend le canal de la box (une seule radio).

10. Démo 4 – Télémétrie MQTT

10.1 Principe

MQTT est un protocole de messagerie « publication / abonnement » : la carte publie des messages sur des **topics** auprès d'un **broker**, et tout client abonné à ces topics les reçoit. Le client MQTT est celui du module ; le STM32 prépare les messages et traite les commandes reçues. Par défaut, le démonstrateur utilise le broker public de test broker.emqx.io (port 1883, sans TLS).

10.2 Démarrage

La carte doit d'abord être connectée à un réseau ayant accès à Internet (démo 2). `mqtt start` se connecte au broker, publie « online » (message retenu), s'abonne aux commandes et publie l'état de la carte toutes les `mqtt.period` secondes :

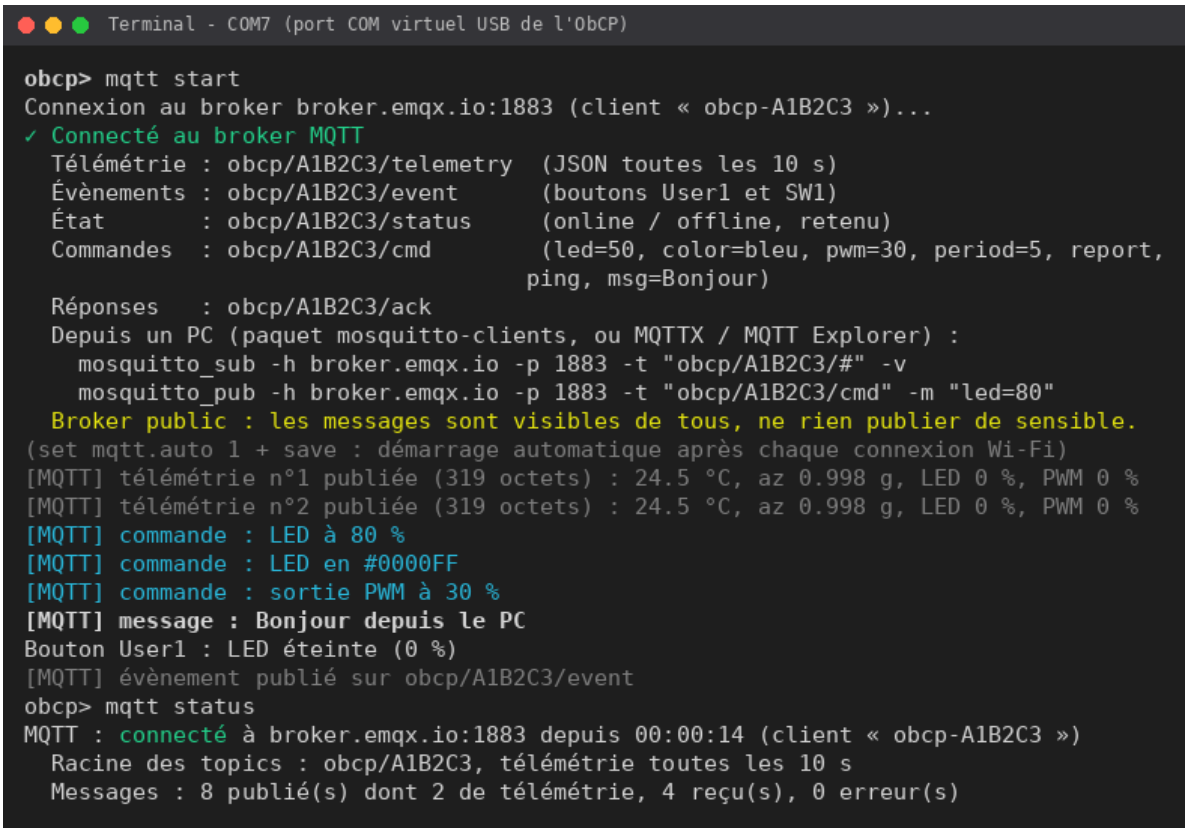


Figure 24 – Connexion au broker, télémétrie, commandes (LED, couleur, PWM, message) et évènement du bouton User1

- `auto mqtt on` (ou `set mqtt.auto 1`) puis `save` : démarrage automatique après chaque connexion Wi-Fi, utile sur batterie, sans terminal (5.6).
- `mqtt trace off` masque l'affichage de chaque publication ; `mqtt status` donne les compteurs ; `mqtt stop` publie « offline » et se déconnecte.
- Le broker publie « offline » à la place de la carte si elle disparaît sans se déconnecter (message de « testament », LWT).
- Si la liaison avec le broker est perdue, le client se reconnecte toutes les 15 s ; après une coupure Wi-Fi, il repart dès que la station est reconnectée.

10.3 Topics et format des messages

Topic (racine obcp/<id>)	Sens	Contenu
…/telemetry	carte → broker	État complet en JSON, toutes les mqtt.period s
…/event	carte → broker	Appui sur User1 ou SW1 : {"event":"button","name":"User1","count":3,"led":0}
…/status	carte → broker	online / offline (retenu)
…/cmd	broker → carte	Commandes (10.4)
…/ack	carte → broker	Réponse à chaque commande : {"cmd":"led","ok":true,"led":80}

Exemple de message de télémétrie :

```
{"id":"A1B2C3","fw":"1.2.0","uptime":95,"temp":24.5,"vdda":3.30,"ax":0.012,"ay":-0.025,
 "az":0.998,"pitch":-0.7,"roll":-1.4,"led":40,"rgb":"#FFFFFF","pwm":0,"user":false,
 "sw1":false,"user_count":0,"sw1_count":0,"rssi":-47,"ssid":"Livebox-7A3C",
 "ip":"192.168.1.42","time":"2026-10-03T08:45:38Z","ap_clients":0,"mqtt":true}
```

Champ	Signification
id, fw, uptime	Identifiant de la carte, version du programme, durée de fonctionnement (s)
temp, vdda	Température de la puce (°C, capteur interne étalonné en usine) et tension d'alimentation analogique (V)
ax, ay, az	Accélérations en g (null si l'accéléromètre est absent)
pitch, roll	Tangage et roulis en degrés (6.3)
led, rgb, pwm	Luminosité (%) et couleur de la LED RVB, rapport cyclique de la sortie PWM (%)
user, sw1, user_count, sw1_count	État courant et nombre d'appuis des boutons User1 (ObCP) et SW1 (X-NUCLEO)
rssi, ssid, ip	Liaison Wi-Fi de la station (rssi vaut null si la carte n'est pas connectée)
time, ap_clients, mqtt	Heure UTC (null si la RTC n'est pas synchronisée), clients du point d'accès, état MQTT

10.4 Commandes à distance

Les messages publiés sur …/cmd peuvent s'écrire led=80, led 80, led:80 ou {"led":80} :

Commande	Effet
led=0…100	Règle la luminosité de la LED RVB
color=<couleur> (ou couleur=, rgb=)	Change la couleur : nom (bleu), #RRGGBB ou R,G,B ; allume la LED si elle était éteinte
pwm=0…100	Règle la sortie PWM du bornier
period=2…3600	Change la période de télémétrie (s) ; save dans le terminal pour la garder
report	Publie immédiatement la télémétrie
ping	Répond sur …/ack avec la durée de fonctionnement (uptime)
msg=<texte>	Affiche le texte dans le terminal de la carte

Une commande inconnue ou une valeur hors limites est signalée sur …/ack par "ok":false.

> **Bon à savoir :** aucune commande « dangereuse » (redémarrage, effacement) n'est acceptée. Sur un broker public, n'importe qui connaissant la racine des topics peut publier sur …/cmd, donc piloter la LED et la sortie PWM : n'y brancher rien qui puisse présenter un danger, ou utiliser un broker privé (10.6).

10.5 Côté PC

Avec les clients Mosquitto (paquet mosquitto-clients sous Linux, installeur Mosquitto sous Windows) :

```
mosquitto_sub -h broker.emqx.io -p 1883 -t "obcp/A1B2C3/#" -v
mosquitto_pub -h broker.emqx.io -p 1883 -t "obcp/A1B2C3/cmd" -m "color=vert"
mosquitto_pub -h broker.emqx.io -p 1883 -t "obcp/A1B2C3/cmd" -m "pwm=50"
mosquitto_pub -h broker.emqx.io -p 1883 -t "obcp/A1B2C3/cmd" -m "msg=Bonjour"
```

Remplacer A1B2C3 par l'identifiant affiché au démarrage. Les outils graphiques MQTTX ou MQTT Explorer permettent aussi de visualiser l'arborescence des topics et de tracer la télémétrie, par exemple l'inclinaison de la carte au cours du temps.

10.6 Utiliser un autre broker

- Broker local : installer Mosquitto sur un PC du réseau (autoriser les connexions anonymes sur le port 1883), puis `set mqtt.host 192.168.1.20`.
- Broker avec authentification : `set mqtt.user …` et `set mqtt.pass …` (identifiant et mot de passe circulent en clair, sans TLS).
- Les connexions chiffrées (TLS, port 8883) ne sont pas prises en charge par cette version du démonstrateur.

> **Attention :** sur un broker public, les messages sont visibles de tous : ne rien publier de sensible.

11. Référence des commandes

Commande	Rôle
`help [commande | demo]`	Aide générale, détail d'une commande, guide rapide
`info`	État de la carte, du module et des services
`config [filtre]` · `set <clé> <valeur>` · `save` · `factory [erase]`	Configuration (5.5)
`auto [web | mqtt | wifi] [on | off]`	Démarrage automatique à la mise sous tension : résumé, ou réglage de `web.auto`, `mqtt.auto` et `wifi.auto` (5.6)
`reboot`	Redémarre la carte (le port COM disparaît un instant)
`led [<0-100> | on | off] [couleur]`	LED RVB : luminosité et couleur (6.1)
`pwm <0-100 | on | off>`	Sortie PWM du bornier J13, 1 kHz (6.2)
`sensors [suivi]`	Température, VDDA, accéléromètre, inclinaisons, boutons (en continu avec suivi)
`time`	Heure UTC et locale de la RTC
`scan [passif] [canal <n>] [tri rssi|canal|ssid] [bssid]`	Démo 1 : balayage
`channels [appliquer]`	Démo 1 : occupation des canaux
`track [ssid] [csv]`	Démo 1 : suivi du RSSI
`connect [ssid [mot de passe]]` · `disconnect` · `status`	Démo 2 : liaison Wi-Fi
`ip [dhcp | <ip> <masque> <passerelle> [dns]]`	Démo 2 : adressage
`dns <nom>` · `ping <hôte> [nombre 1-50] [intervalle 100-3500 ms]` · `ntp [serveur]`	Démo 2 : services réseau
`web start | stop | status`	Démo 3 : point d'accès et serveur web
`mqtt start | stop | status | pub <topic> <message> | sub <topic> | trace on|off`	Démo 4 : MQTT (topic relatif à la racine, sauf s'il commence par `/` ; le message peut contenir des espaces)
`at <commande>` (ou directement `AT+…`)	Commande AT brute envoyée au module
`log on | off`	Trace du trafic AT (mots de passe masqués)
`stats`	Compteurs SPI et AT
`dfu`	Redémarre l'ObCP dans le chargeur USB DFU, après confirmation (4.1)
`module [reset | ota | boot]`	Versions du module, redémarrage, mise à jour par le terminal (3.3), première programmation par la passerelle de l'ObCP (3.4)

Commandes AT utiles pour explorer le module (`log on` montre celles qu'utilisent les démos) :

```
AT+GMR            versions du firmware
AT+CWSTATE?       état de la station
AT+CWJAP?         réseau, BSSID, canal, RSSI
AT+CIPSTA?        adresse IP de la station
AT+CIPSNTPTIME?   heure du module
```

12. Dépannage

Premier réflexe : la commande `info` résume l'état de la carte, du module, de la liaison Wi-Fi et des services.

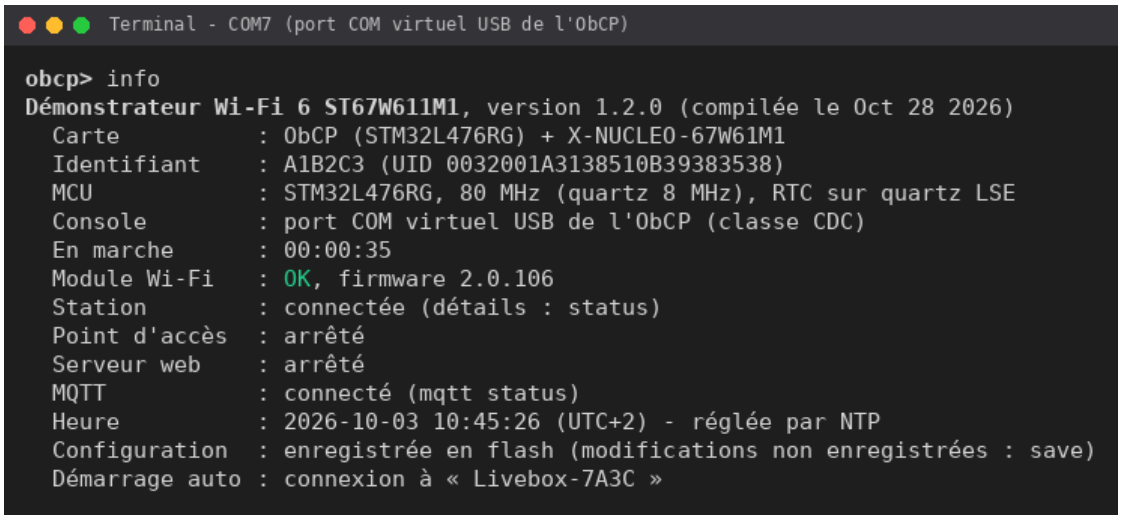


Figure 25 – Résumé de l'état de la carte (commande info)

Symptôme	Cause probable	Solution
Aucun port COM n'apparaît	Câble de charge seulement, SW2 sur OFF, carte non programmée, ou SW4 sur Prog	Câble de données, SW2 sur ON (LED PWR ON) ; programmer par DFU (4.2) ; SW4 sur Run puis RST
LED rouge : 2 éclats répétés, pas de port COM	Le quartz 8 MHz (HSE) ne démarre pas : programme arrêté dans `Error_Handler()`	Vérifier le quartz 8 MHz et ses condensateurs (soudures)
LED rouge : 3 éclats répétés (après une dizaine de secondes), pas de port COM	Le quartz 32,768 kHz (LSE) ne démarre pas	Vérifier le quartz 32 kHz et ses condensateurs (soudures). En attendant : dans CubeMX, **Clock Configuration › RTC Clock Mux** sur LSI et **Timers › RTC › Synchronous Predivider value** à 249 (32 kHz / 128 / 250 = 1 Hz), puis régénérer ; démarrage retardé de 5 s, heure moins précise entre deux mises à l'heure NTP
LED rouge : clignotement rapide continu	Autre erreur d'initialisation (configuration modifiée dans CubeMX ?)	Revenir au `.ioc` d'origine ; une sonde SWD sur J17 montre l'appel à `Error_Handler()`
Après une régénération : erreurs de compilation	Accroche effacée, étiquette de broche renommée, périphérique désactivé, gestionnaires de faute générés (« multiple definition of HardFault_Handler »)	`python Tools/verif_cubemx.py` indique la cause (4.4)
« STM32 BOOTLOADER » au lieu du port COM	La carte est dans le chargeur DFU (SW4 sur Prog, ou après `dfu`)	Programmer (4.2), ou SW4 sur Run puis RST
Le port COM disparaît	Reset, `reboot`, `dfu`, faute du programme, coupure	Rouvrir la connexion du terminal (5.1)
Terminal ouvert mais rien ne s'affiche	Bannière déjà lue, mauvais port	Taper Entrée ou `help` ; vérifier le port (Gestionnaire de périphériques)

Symptôme	Cause probable	Solution
Linux : port occupé juste après le branchement, ou lignes « AT… » inattendues	ModemManager sonde le nouveau port série pendant quelques secondes	Attendre, ou désactiver ModemManager : `sudo systemctl disable --now ModemManager`
STM32CubeProgrammer : « No DFU detected »	Chargeur non actif, pilote DFU absent	SW4 sur Prog et RST (ou `dfu`) ; installer STM32CubeProgrammer avec son pilote (4.1)
Après programmation, rien ne démarre	SW4 resté sur Prog	SW4 sur Run, puis RST
Accents ou barres illisibles	Terminal non réglé en UTF-8	Régler le jeu de caractères en UTF-8 (5.2)
Codes du type [32m visibles	Terminal sans couleurs ANSI	`set term.color 0` puis `save`
« accéléromètre absent ou non reconnu »	WHO_AM_I = 0x00 ou 0xFF : le capteur ne répond pas ; autre valeur : modèle inconnu	Vérifier le composant U4 et ses soudures ; le reste du démonstrateur fonctionne
La LED RVB reste éteinte	Luminosité à 0	`led blanc 100` ; vérifier ensuite chaque couleur (`led rouge`, `led vert`, `led bleu`)
La charge du bornier PWM ne réagit pas	Branchement, charge trop puissante	Charge entre +5 V et PWM (2.5), `pwm 100` ; 2,5 W au plus
Redémarrages inopinés sur batterie	Batterie faible : pointes de courant du Wi-Fi	Recharger, ou alimenter par l'USB
Sur batterie, le point d'accès ObCP-WiFi-<id> n'apparaît pas	`web.auto` à 0, réglage non enregistré (`save` oublié), ou démarrage avec User1 appuyé (valeurs d'usine)	Avec le terminal : `auto` affiche ce qui démarre seul ; `auto web on` puis `save` (5.6)
Sur batterie, pas de télémétrie MQTT	`mqtt.auto` à 0, aucune box enregistrée (ou `wifi.auto` à 0), box hors de portée ou sans accès Internet	`auto` (5.6) ; essayer d'abord avec le terminal : `connect`, puis `mqtt start`
« le module ne répond pas »	JP1/JP2 absents, carte mal enfichée, firmware	Vérifier le montage (2.3), puis `module reset` ; sinon première programmation (3.4)
« magic 0x5501 » ou « firmware T02 détecté »	Firmware du module trop ancien ou d'une autre architecture	Première programmation en mission T01 v2.0.106 (3.4)
`module boot` : QConn_Flash échoue	Terminal resté ouvert, mauvais port, fichier de configuration	User1 pour quitter, fermer le terminal, vérifier `cfg_obcp.ini`, recommencer (3.4) ; sinon 3.5
Après QConn_Flash, le terminal rouvert reste muet	Passerelle toujours active (une touche a été tapée)	Appuyer sur User1 : la passerelle s'arrête et le module redémarre
Réseau absent du scan	Réseau 5 GHz, SSID masqué, trop loin	Activer le 2,4 GHz ; essayer `scan passif`
Échec de connexion	Mot de passe, sécurité WPA-Entreprise	Voir la cause affichée (8.1)
Pas d'heure NTP	UDP 123 filtré, pas d'accès Internet	Autre réseau ou `set time.server <serveur local>`
`mqtt start` : « connexion au broker impossible »	Port 1883 filtré, broker indisponible, pas d'accès Internet	Partage de connexion d'un téléphone, broker local (10.6)
La page web ne s'ouvre pas	Le téléphone passe par les données mobiles	Rester connecté au réseau sans Internet, couper les données mobiles

Symptôme	Cause probable	Solution
Pas de QR code	Couleurs désactivées	`set term.color 1` (le QR code utilise les couleurs ANSI)
Heure locale décalée	Fuseau ou heure d'été	`set time.tz 60`, `set time.dst 1`
`module ota` : des « C » s'affichent, rien ne se passe	Envoi YMODEM non lancé (délai de 2 min)	Tera Term : Fichier › Transfert › YMODEM › Envoyer (File › Transfer › YMODEM › Send) ; PuTTY ne sait pas envoyer de fichier : script fourni (3.3)
`module ota` : « fichier .ota attendu », « image T02 »	Mauvais fichier	Prendre `st67w611m_mission_t01_vX.Y.Z.xz.ota` (3.3)
`module ota` : « image refusée par le module »	Fichier altéré, ou version plus ancienne	Télécharger à nouveau le fichier ; le module garde son firmware

Pour aller plus loin dans le diagnostic : `log on` affiche les commandes AT envoyées (->) et les réponses ou évènements reçus (<-), `stats` les compteurs d'erreurs SPI (en-têtes invalides, délais dépassés, renvois rx_stall). Si la console USB elle-même ne fonctionne pas, une sonde SWD sur J17 permet de suivre le programme pas à pas (4.3).

13. Pour aller plus loin

Le code de la démo est commenté pour un lecteur débutant : chaque fichier commence par un résumé de son rôle, et chaque fonction publique est décrite dans son en-tête (`.h`). La **documentation du programme** (`Doc/Documentation_Programme_ObCP.docx`) en donne la vue d'ensemble : carte des fichiers et de leurs dépendances, schémas de déroulement (démarrage, boucle principale, trajet d'une commande, d'une requête web, d'un message MQTT), dialogue avec le module, mémoire, brochage, référence des fonctions, et recettes détaillées pour plusieurs des exercices ci-dessous.

Quelques pistes d'exercices à partir du code fourni :

- **Ajouter une commande** au terminal : une entrée dans la table `s_cmds` de `Core/Src/app/cli.c` et une fonction `int cmd_xxx(int argc, char **argv)`.
- **Ajouter un périphérique avec CubeMX** (entrée analogique sur A0, bus I2C pour un capteur, sortie supplémentaire…) : le configurer dans `ObCP_WiFi_Demo.ioc`, régénérer, puis l'utiliser depuis ses propres fichiers (4.4).
- **Ajouter une mesure** : la lire dans `io_read()` et l'ajouter au JSON de `app_status_json()` ; elle apparaît alors dans la page web et dans la télémétrie MQTT. L'accéléromètre peut par exemple détecter un choc ou une chute libre (seuils sur la norme de l'accélération).
- **Asservir la sortie PWM** : faire tourner un ventilateur selon la température de la puce, ou un moteur selon l'inclinaison de la carte.
- **Ajouter une route HTTP** : un test sur `req->path` dans la fonction `handle()` de `demo_web.c`.
- **Ajouter une commande MQTT** : une branche dans `handle_cmd()` de `demo_mqtt.c`.
- **Utiliser le module Bluetooth LE de l'ObCP** (BlueNRG-M0, sur SPI2), aujourd'hui maintenu en reset, en reprenant le programme modèle du dépôt ObCP.
- **Explorer le module** avec les commandes AT (`log on`, `at …`) : économie d'énergie, Bluetooth LE du ST67W611M1, MQTT sur TLS avec certificats.
- **Mise à jour par le Wi-Fi** : faire télécharger l'image .ota par le module depuis un serveur HTTP du PC (`python -m http.server`), comme l'exemple FOTA de ST, et la transmettre avec les fonctions de `ncp_fwu.c` utilisées par `module ota`.
- **Tester sur PC** : `make -C Tests` exécute les tests et produit `Tests/build/sim_transcript.txt`, le journal d'une session complète des quatre démos face au module simulé.

Annexe A – Organisation du logiciel

Vue d'ensemble des fichiers ; la documentation du programme (`Doc/Documentation_Programme_ObCP.docx`) détaille chacun d'eux : rôle, fonctions, fichiers qui l'utilisent.

Fichier	Rôle
`ObCP_WiFi_Demo.ioc`	Configuration STM32CubeMX : broches et étiquettes, horloges, périphériques, NVIC, options du projet
`Core/Src/main.c` (généré)	`HAL_Init()`, `SystemClock_Config()` (80 MHz), `MX_xxx_Init()` ; accroches USER CODE vers `board.c` et `app.c` (4.4)
`gpio.c`, `spi.c`, `adc.c`, `tim.c`, `rtc.c`, `stm32l4xx_hal_msp.c` (générés)	Broches, SPI1, SPI3, ADC1, TIM1, TIM3, RTC et leurs fonctions MSP (horloges des périphériques, broches)
`Core/Src/stm32l4xx_it.c` (généré)	SysTick, USB, NMI (accroche vers `fault_nmi()`)
`USB_DEVICE/` (généré)	Descripteurs USB (numéro de série tiré de l'identifiant unique), démarrage de l'USB, liaison avec la HAL ; rappels CDC transmis à `cdc_port.c`
`Core/Src/board.c`	Saut dans le chargeur DFU, quartz 32 kHz, fin de l'initialisation (SPI, PWM, ADC, date de la RTC), diagnostic par la LED rouge, identifiant unique, rapport de faute, heure UTC
`Core/Src/fault_handlers.c`	HardFault, MemManage, BusFault, UsageFault (assembleur) et NMI : adresse fautive conservée dans la RTC, LED rouge clignotante puis redémarrage
`Core/Src/cdc_port.c`	Port COM virtuel : tampons circulaires (4 Ko en émission, 2 Ko en réception), contrôle de flux, ouverture du terminal (DTR)
`Core/Src/ncp_port_obcp.c`	Adaptation du pilote : CS, RDY, CHIP_EN, BOOT, transfert SPI, temps
`Core/Src/uart_bridge.c`, `app/bridge_ctl.c`	Passerelle USB ↔ UART du module pour `module boot` (USART2 et DMA par registres) ; décision de fin de session, testée sur PC
`Core/Src/app/app.c`	Initialisation (bannière, démarrage automatique), boucle principale, tâches de fond, messages d'erreur, JSON d'état
`Core/Src/app/console.c`	Console sur le port COM virtuel, éditeur de ligne, évènements asynchrones, réouverture du terminal
`Core/Src/app/cli.c`	Table des commandes, aide, configuration, LED, PWM, capteurs, DFU, outils AT
`Core/Src/app/config.c`, `cfg_codec.c`	Paramètres (accès par clé, contrôles), stockage en flash avec CRC32
`Core/Src/app/io.c`, `color.c`, `lis3dh.c`	LED RVB (couleurs, correction gamma), sortie PWM, boutons (anti-rebond), ADC (moyenne de 16 conversions), accéléromètre LIS3DH ou LIS3DSH et inclinaisons
`Core/Src/app/demo_*.c`, `web_page.c`, `http_parse.c`, `qr_term.c`	Les quatre démonstrations
`Core/Src/app/ymodem.c`, `module_ota.c`	Réception YMODEM (CRC 16 bits, blocs de 128 ou 1024 o) et commande `module ota`
`Core/Src/st67w6x/ncp_spi.c`	Trames SPI : en-tête, rx_stall, lecture du reliquat, acquittement
`Core/Src/st67w6x/ncp_at.c`	Commandes AT, analyse du flux (lignes, données binaires, invite >), évènements
`ncp_wifi.c`, `ncp_net.c`, `ncp_mqtt.c`, `ncp.c`	Fonctions Wi-Fi, réseau, MQTT et démarrage du module

Fichier	Rôle
`Core/Src/st67w6x/ncp_fwu.c`	Mise à jour du firmware du module : en-tête .ota, AT+OTASTART, AT+OTASEND, AT+OTAFIN

Fonctionnement sans RTOS. Les fonctions du pilote sont bloquantes avec délai de garde : pendant l'attente d'une réponse, les trames reçues continuent d'être traitées et les évènements non sollicités (+CW:, +CIP:, +IPD:, +MQTT:) mettent à jour des drapeaux. La boucle principale (`app_loop()`, appelée depuis `main.c`, et `app_idle()`) exploite ensuite ces drapeaux : reconnexion, NTP, requêtes HTTP, publications MQTT, boutons. Les commandes longues appellent elles aussi `app_idle()` pour que le serveur web et MQTT restent actifs. L'USB est traité par interruption : la console n'écrit et ne lit que dans des tampons circulaires.

Annexe B – Protocole SPI du ST67W611M1

Chaque échange SPI (mode 0, 8 bits) commence par un en-tête de 8 octets, dans les deux sens, suivi de la charge utile ; la longueur totale est arrondie au multiple de 4 supérieur.

Octets	Contenu
0–1	Magic 0x55AA (petit-boutiste : AA 55)
2–3	Longueur de la charge utile (petit-boutiste)
4	Bits 0–1 : version (0) ; bit 2 : rx_stall (le récepteur n'a pas pu accepter la trame) ; bits 3–7 : drapeaux
5	Type de trafic (0 = commandes AT)
6–7	Réservé (0)

1. L'hôte lève CS (actif à l'état haut) et attend RDY = 1 ; si RDY est déjà à 1, c'est le module qui demande un échange.
2. Échange full-duplex des en-têtes et des charges utiles : l'hôte lit l'en-tête du module pendant qu'il émet le sien.
3. Si la trame du module est plus longue que ce que l'hôte a émis, l'hôte lit le reliquat en émettant des zéros.
4. Le module abaisse RDY pour acquitter ; l'hôte abaisse CS. Si rx_stall était à 1, la trame de l'hôte est réémise plus tard.

Annexe C – Tests sur PC

`make -C Tests` (ou `make test`) compile avec gcc et exécute cinq programmes, avec les vérifications d'accès mémoire (AddressSanitizer) et de comportement indéfini (UBSan) :

- **test_driver** : le pilote face à un module simulé (trames SPI, rx_stall, trames fragmentées, données binaires, invite d'envoi, MQTT, mise à jour du module avec vérification SHA-256 de l'image reçue).
- **test_app** : calendrier et heure d'été, analyse des adresses et nombres, JSON, analyse HTTP, configuration et stockage (dont la relecture d'une configuration enregistrée par la version 1.1), réception YMODEM face à un émetteur simulé (CRC faux, ACK perdu, annulations, délais), couleurs de la LED, accéléromètres LIS3DH et LIS3DSH simulés (protocole SPI, conversions, inclinaisons), fin de session de la passerelle `module boot` (QConn_Flash qui garde DTR à 0, effacement silencieux, terminal rouvert, bouton).
- **test_usb** : le port COM virtuel (`cdc_port.c` et les rappels générés de `usbd_cdc_if.c`) face à une classe CDC simulée (tampons circulaires, découpage des envois, contrôle de flux en réception, ouverture et fermeture du terminal, veille de l'hôte, `CDC_Transmit_FS()`).

- **test_sim** : l'application complète (console, commandes, quatre démos, LED, PWM, capteurs, `dfu`, `module ota`, `module boot`, reprises sur incident) ; le journal de la session est écrit dans `Tests/build/sim_transcript.txt`, et les réponses JSON et le JavaScript de la page sont validés.
- **test_boot** : trois mises sous tension avec une configuration déjà enregistrée en flash, comme une carte sur batterie (box, serveur web et MQTT ; serveur web sans box ; box qui refuse la connexion), chacune dans une exécution séparée : messages de la bannière, ordre des opérations (point d'accès après la connexion à la box), absence de QR code, nouvelles tentatives de connexion.
- **Essai de bout en bout** (si Python 3 et pyserial sont présents) : l'application tourne en temps réel sur un pseudo-terminal et reçoit des images .ota envoyées par `Tools/maj_module_ota.py`, puis par `sz --ymodem` si lrzsz est installé.
- **Tools/verif_cubemx.py** : accroches du code de la démo dans les fichiers générés, étiquettes des broches, périphériques et gestionnaires de faute (la vérification à refaire après chaque régénération, 4.4).

Les captures de terminal et de la page web de ce guide proviennent de cette simulation, avec un jeu de données fictif (noms de réseaux, adresses, mesures). Le matériel propre à l'ObCP (USB, DFU, passerelle, accéléromètre) y est simulé : ces tests ne remplacent pas l'essai sur carte (1.5).

Annexe D – Références et licences

- Dépôt de l'ObCP (SUPMICROTECH-ENSMM) : documents de fabrication (schéma V2.3, nomenclature, implantation), programme modèle, procédure de programmation par USB DFU. https://github.com/jimbaud/LAB_ObCP_L476_STM32CubeIDE
- UM3449 – X-NUCLEO-67W61M1 user manual (STMicroelectronics) : brochage Arduino, cavaliers, ponts de soudure. https://www.st.com/resource/en/user_manual/um3449-wifibluetooth802154-connectivity-expansion-board-based-on-the-st67w611m1-module-for-stm32-nucleo-boards-stmicroelectronics.pdf
- X-CUBE-ST67W61 v1.3.0 (STMicroelectronics) : pilote officiel, binaires du module, scripts de mise à jour (UM3475 : Getting started with X-CUBE-ST67W61). https://www.st.com/en/embedded-software/x-cube-st67w61.html
- Dépôt GitHub de X-CUBE-ST67W61 : binaires du module (dossier NCP_Binaries), outil QConn_Flash, scripts. https://github.com/STMicroelectronics/x-cube-st67w61/tree/main/Projects/ST67W6X_Scripts/Binaries
- Wiki ST : projet FOTA du ST67W611M1 (mise à jour du module par le lien SPI). https://wiki.st.com/stm32mcu/wiki/Connectivity:Wifi_ST67W6X_FOTA_Application
- Wiki ST : interface SPI du ST67W611M1 et installation matérielle. https://wiki.st.com/stm32mcu/wiki/Connectivity:Wi-Fi_spi
- AN2606 – STM32 microcontroller system memory boot mode (STMicroelectronics) : chargeur en ROM et interface USB DFU. https://www.st.com/resource/en/application_note/an2606-stm32-microcontroller-system-memory-boot-mode-stmicroelectronics.pdf
- STM32CubeProgrammer (STMicroelectronics) : programmation par USB DFU, outil en ligne de commande STM32_Programmer_CLI. https://www.st.com/en/development-tools/stm32cubeprog.html
- STM32 USB Device Library (STMicroelectronics) : cœur et classe CDC utilisés pour le port COM virtuel. https://github.com/STMicroelectronics/stm32-mw-usb-device
- Forum ST Community : « ST67W611M1 X-NUCLEO-67W61M1 on NUCLEO-H753ZI: SPI magic mismatch 0x5501 » (firmware d'usine trop ancien). https://community.st.com/interface-and-connectivity-ics-52/st67w611m1-x-nucleo-67w61m1-on-nucleo-h753zi-spi-magic-mismatch-0x5501-unclear-if-firmware-flashed-qconn-flash-lock-question-168072
- UM1718 – STM32CubeMX for STM32 configuration and initialization C code generation (STMicroelectronics) : sections USER CODE, options de génération. https://www.st.com/resource/en/user_manual/um1718-stm32cubemx-for-stm32-configuration-and-initialization-c-code-generation-stmicroelectronics.pdf
- ST Community : « STM32CubeIDE 2.0.0 workflow tutorial » (génération du code par STM32CubeMX autonome, actualisation du projet). https://community.st.com/t5/stm32-mcus/stm32cubeide-2-0-0-workflow-tutorial/ta-p/864831
- QR Code generator library (Project Nayuki). https://www.nayuki.io/page/qr-code-generator-library

Composant	Licence
Application (`Core/Src/app/`), fichiers de la démo dans `Core/`, pilote `Core/Src/st67w6x`	Code de ce paquet ; le pilote reprend le protocole du pilote ST (BSD-3-Clause, voir `Core/Src/st67w6x/LICENSE.md`)
Fichiers générés par STM32CubeMX dans `Core/`, HAL STM32L4	BSD-3-Clause (STMicroelectronics), voir `Core/LICENSE_ST.md`

Composant	Licence
STM32 USB Device Library et fichiers générés de `USB_DEVICE/`	SLA0044 « Ultimate Liberty » (STMicroelectronics) : redistribution autorisée, utilisation sur microcontrôleurs ST ; texte dans `Middlewares/ST/STM32_USB_Device_Library/LICENSE.txt`
CMSIS (ARM, ST)	Apache 2.0
QR-Code-generator (Project Nayuki)	MIT